

Département Génie Électrique

Rapport de Projet de Fin d'Année

Filière : Cycle Ingénieur en Génie Électrique et Contrôle Industriel (GECI)

SKYPHARMA

**Conception et Réalisation d'un Système de Pharmacie Autonome
(Robot ASRS) avec Intégration d'un Système de Livraison par Drone**

Réalisé par :

- M. BADER EL BERKAOUI
- M. MOHAMMED HSINY

Encadré par :

Pr. Nada El Gmili

Dédicaces

À nos très chers parents, Aucune dédicace ne saurait exprimer le respect, l'amour éternel et la gratitude que nous vous portons. Vous qui avez toujours été nos piliers, sacrifiant tant pour notre éducation et notre réussite. Que ce travail soit le témoignage de notre profonde reconnaissance pour vos prières quotidiennes, votre patience et votre soutien indéfectible. Que Dieu vous accorde santé, longue vie et bonheur.

À nos familles, frères et sœurs, Pour votre présence constante, vos encouragements et votre affection qui nous ont portés tout au long de notre parcours universitaire.

À nos chers amis et à tous ceux qui nous sont chers, En souvenir des moments partagés, des défis relevés ensemble au sein de notre promotion, et pour votre soutien précieux.

Remerciements

Avant tout, nous tenons à rendre grâce à Dieu, le Tout-Puissant, de nous avoir donné la force, le courage et la patience nécessaires pour mener à bien ce projet d'envergure.

Nous exprimons notre profonde gratitude et nos sincères remerciements à notre encadrante, Pr. Nada El Gmili, pour sa disponibilité, sa rigueur scientifique et la qualité de son suivi pédagogique. Ses conseils avisés, son soutien technique précieux et ses encouragements constants ont été le moteur indispensable à la réussite du projet SkyPharma. Qu'elle trouve ici l'expression de notre très grand respect et de notre reconnaissance.

Nos vifs remerciements s'adressent également aux membres du jury qui nous font l'honneur d'évaluer notre travail et d'enrichir ce rapport par leurs remarques constructives.

Enfin, nous tenons à remercier chaleureusement l'ensemble du corps professoral et administratif de la Faculté des Sciences et Techniques de Mohammedia (FSTM), et plus particulièrement les enseignants du département Génie Électrique de la filière Génie Électrique et Contrôle Industriel (GECI), pour la qualité de la formation qu'ils nous ont dispensée tout au long de notre cursus d'ingénieur.

À tous ceux qui ont contribué de près ou de loin à la réalisation de ce travail, nous vous disons un grand merci.

Résumé

Ce projet présente la conception, la réalisation et l'automatisation de SkyPharma, un écosystème logistique de santé innovant combinant un système de stockage et de distribution automatisé (robot ASRS) et un vecteur de livraison par drone autonome. L'objectif principal est d'optimiser la gestion des flux pharmaceutiques en milieu hospitalier ou industriel, et de sécuriser la distribution de médicaments urgents vers des zones enclavées.

Le cœur du système repose sur un robot cartésien à trois axes (X, Y, Z) piloté numériquement par le micrologiciel GRBL chargé sur une carte Arduino. Une armoire électrique dédiée a été dimensionnée et câblée pour assurer la gestion de la puissance, le contrôle des moteurs pas-à-pas et la sécurité du système via des capteurs de fin de course. Côté logiciel, une application développée en Python orchestre l'ensemble : elle intègre une base de données relationnelle pour la gestion des stocks et la cartographie des alvéoles de stockage, ainsi qu'une interface graphique de supervision. Cette interface permet aux clients de suivre le déplacement estimé du robot en temps réel par télémétrie calculée, et offre aux administrateurs des outils avancés de maintenance et de tests unitaires pour les capteurs. Une fois le médicament extrait par le robot ASRS, un mécanisme de transfert assure son embarquement sur un drone multirotor configuré pour une navigation aérienne autonome vers sa destination finale.

Les tests unitaires et d'intégration réalisés valident la robustesse de l'armoire électrique, la précision du positionnement mécanique, ainsi que la réactivité du système face aux alarmes de sécurité. Ce projet pluridisciplinaire met en évidence l'apport des technologies de l'automatisme, de l'informatique industrielle et de la robotique mobile dans la modernisation de la logistique médicale.

Mots-clés : SkyPharma, Stockage Automatisé (ASRS), GRBL, Arduino, Python, Interface de Supervision, Drone Autonome, Génie Électrique.

Abstract

This project introduces the design, development, and automation of SkyPharma, an innovative healthcare logistics ecosystem that combines an Automated Storage and Retrieval System (ASRS robot) with an autonomous drone delivery platform. The primary goal is to optimize pharmaceutical inventory management within hospital or industrial settings and secure the distribution of emergency medical supplies to remote or isolated areas.

The core of the hardware system relies on a 3-axis (X, Y, Z) Cartesian robot digitally controlled by the GRBL firmware flashed onto an Arduino board. A custom-designed electrical cabinet was dimensioned and wired to handle power distribution, stepper motor control, and system safety through hardware limit switches. On the software side, an application developed in Python orchestrates the entire workflow. It incorporates a relational database for inventory management and slot mapping, alongside a dual-purpose graphical user interface (GUI). This interface allows clients to track the estimated movement of the robot in real time via dead reckoning calculations, while providing administrators with advanced diagnostic tools for unit testing and sensor maintenance. Once the medicine is retrieved by the ASRS robot, a transfer mechanism loads the payload onto a multirotor drone configured for autonomous aerial navigation to its destination.

The unit and integration tests successfully validate the reliability of the electrical cabinet, the accuracy of the mechanical positioning, and the system's responsiveness to safety alarms. This multidisciplinary project highlights the significant impact of automation, industrial computing, and mobile robotics on modernizing medical logistics.

Keywords: SkyPharma, Automated Storage and Retrieval System (ASRS), GRBL, Arduino, Python, Supervision GUI, Autonomous Drone, Electrical Engineering.

Table des matières

Dédicaces	i
Remerciements	ii
Résumé	iii
Abstract	iv
Liste des abréviations	xii
INTRODUCTION GÉNÉRALE	1
Chapitre 1 : État de l’art et étude des solutions existantes	4
1.1 Introduction	4
1.2 Problématique d’accès aux médicaments dans les zones isolées et en situation d’urgence	4
1.3 Robotisation des pharmacies	5
1.4 Robots ASRS industriels	6
1.5 Pharmacies intelligentes	7
1.6 Livraison intelligente des médicaments	7
1.7 Technologies utilisées dans Sky Pharma	8
1.8 Conclusion	8
Chapitre 2 : Analyse fonctionnelle et conception globale.....	10
2.1 Introduction	10
2.2 Présentation générale de Sky Pharma.....	10
2.2.1 Vision du projet	10
2.2.2 Principe général	10
2.3 Modes de distribution et de livraison.....	12
2.4 Analyse du besoin.....	12
2.4.1 Expression du besoin	12
2.4.2 Bête à cornes.....	13
2.5 Outils d’analyse fonctionnelle	14
2.5.1 Fonctions principales	14
2.5.2 Fonctions contraintes	14
2.5.3 Diagramme FAST.....	15
2.5.4 Diagramme SADT	15
2.6 Cahier des charges fonctionnel.....	16
2.6.1 Exigences fonctionnelles	17

2.6.2 Exigences techniques	17
2.6.3 Contraintes.....	17
2.7 Architecture globale du système	18
2.8 Architecture des flux.....	20
2.9 Diagrammes UML.....	20
2.9.1 Diagramme de cas d'utilisation	20
2.9.2 Diagramme de séquence.....	21
2.10 Gestion des erreurs, sécurité et justification.....	22
2.11 Conclusion.....	24
Chapitre 3 : Conception mécanique et électronique du robot ASRS	25
3.1 Introduction.....	25
3.2 Architecture mécanique générale	25
3.3 Structure porteuse.....	26
3.4 Guidage et éléments linéaires.....	27
3.5 Supports et pièces de fixation.....	28
3.6 Chariot mobile et transmission H-Bot.....	29
3.7 Assemblage CAO du système	30
3.8 Points de validation mécanique.....	32
3.9 Synthèse de la conception mécanique	32
3.10 Architecture électronique et commande	33
3.10.1 Architecture électronique globale.....	33
3.10.2 Carte Arduino UNO.....	34
3.10.3 CNC Shield V3.....	35
3.10.4 Drivers DRV8825.....	36
3.11 Motorisation, capteurs et alimentation.....	37
3.11.1 Fins de course	37
3.11.2 Alimentation électrique.....	38
3.11.3 Gestion thermique.....	39
3.12 Schéma électrique et câblage de l'armoire.....	40
3.13 Validation électromécanique	40
3.14 Conclusion.....	42
Chapitre 4 : Développement embarqué et pilotage du robot.....	43
4.1 Introduction.....	43
4.2 Rôle de la partie embarquée dans Sky Pharma	43

4.3 Choix du firmware GRBL.....	45
4.3.1 Présentation de GRBL.....	45
4.3.2 Justification du choix	45
4.3.3 Limites de GRBL.....	46
4.4 Installation et configuration initiale de GRBL	46
4.4.1 Installation du firmware	46
4.4.2 Communication série	46
4.5 Organisation des axes du robot	47
4.5.1 Axes logiques	47
4.5.2 Axes physiques	47
4.5.3 Architecture H-Bot.....	47
4.5.4 Approche adoptée	48
4.6 Système de coordonnées	48
4.7 Calcul des steps/mm et intégration avec la plateforme Python.....	49
4.7.1 Définition.....	49
4.7.2 Exemple courroie GT2.....	49
4.7.3 Axe Z.....	50
4.7.4 Correction.....	50
4.8 Paramètres GRBL principaux	50
4.9 Homing et référencement.....	50
4.10 Génération automatique du G-code.....	51
4.11 Communication série avec Python	51
4.12 Sécurité embarquée.....	52
Chapitre 5 : Développement logiciel, base de données et interfaces de commande	54
5.1 Introduction.....	54
5.2 Rôle de la partie logicielle dans Sky Pharma.....	54
5.3 Architecture logicielle globale	55
5.4 Choix techniques adoptés.....	56
5.5 Base de données SQLite	57
5.6 Organisation des tables de la base de données	58
5.7 Interface client distante.....	58
5.8 Traitement logiciel après confirmation d'une commande	59
5.9 Génération automatique du G-code.....	60
5.10 Lien entre le logiciel et le robot.....	61

5.11 Interface locale de récupération.....	62
5.12 Processus de récupération locale	62
5.13 Gestion du stock et traçabilité	63
5.14 Validation logicielle et amélioration par intelligence artificielle	64
Chapitre 6 : Réalisation du système de livraison par drone	65
6.1 Introduction	65
6.2 Objectif du système de livraison par drone.....	65
6.3 Architecture générale du module drone.....	66
6.4 Choix de la plateforme aérienne.....	67
6.4.1 Châssis F450.....	67
6.4.2 Contrôleur de vol	67
6.4.3 Radiocommande FlySky FS-i6.....	68
6.4.4 Motorisation.....	69
6.4.5 Hélices.....	70
6.4.6 Batterie	70
6.5 Mécanisme d'accrochage et de largage du médicament	71
6.5.1 Objectif du mécanisme.....	71
6.5.2 Principe mécanique	71
6.5.3 États du mécanisme.....	72
6.5.4 Intégration mécanique	72
6.5.5 Commande du servomoteur.....	73
6.5.6 Séquence de livraison avec largage	73
6.6 Architecture électronique du drone	75
6.7 Principe de fonctionnement de la mission	76
6.8 Sécurité du vol.....	76
6.9 Validation expérimentale	77
6.10 Limites et perspectives.....	77
6.11 Conclusion.....	78
CONCLUSION GÉNÉRALE	79
ANNEXES ET BIBLIOGRAPHIE.....	80
Annexe A : Sources et références consultées.....	80

Table des figures

Figure 1.1 - Contexte d'une officine et limites de distribution traditionnelle	19
Figure 1.2 - Références visuelles de systèmes automatisés de pharmacie	20
Figure 2.1 - Fonctionnement global de SkyPharma	24
Figure 2.2 - Bête à cornes du système SkyPharma	27
Figure 2.3 - Diagramme pieuvre de SkyPharma	27
Figure 2.4 - Diagramme FAST de SkyPharma	28
Figure 2.5 - Diagramme SADT A-0 de SkyPharma.....	29
Figure 2.6 - Cartographie technologique du prototype SkyPharma	32
Figure 2.7 - Architecture globale du système SkyPharma.....	33
Figure 2.8 - Modèle conceptuel de données de SkyPharma	34
Figure 2.9 - Diagramme de cas d'utilisation de SkyPharma	35
Figure 2.10 - Diagramme de séquence du cycle de commande	35
Figure 3.1 - Principe général de l'architecture H-Bot retenue pour le robot ASRS	38
Figure 3.2 - Châssis porteur modélisé sous SolidWorks.....	39
Figure 3.3 - Profilé V-Slot utilisé pour le guidage du chariot.....	40
Figure 3.4 - Tige cylindrique utilisée comme élément de guidage.....	40
Figure 3.5 - Roulement linéaire utilisé pour réduire les frottements de translation	41
Figure 3.6 - Support mécanique de guidage et de fixation	41
Figure 3.7 - Pièce de fixation utilisée pour l'intégration des éléments de guidage	42
Figure 3.8 - Détail du chariot monté sur le profilé de guidage.....	42
Figure 3.9 - Liaison entre tige de guidage, support et chariot mobile.....	43
Figure 3.10 - Vue d'ensemble de l'assemblage mécanique du robot.....	44
Figure 3.11 - Vue latérale de l'assemblage avec guidage et supports	44
Figure 3.12 - Vue de dessous montrant l'intégration des profilés et du guidage.....	45
Figure 3.13 - Architecture électrique détaillée avec alimentation, drivers, capteurs et axes	46

Table des figures (suite)

Figure 3.14 - Architecture électronique globale de SkyPharma	47
Figure 4.1 - Chaîne de pilotage embarquée du logiciel vers le mouvement robot	57
Figure 4.2 - Architecture du développement embarqué	58
Figure 5.1 - Architecture logicielle globale de SkyPharma	68
Figure 5.2 - Chaîne logicielle de commande, base de données et traçabilité.....	68
Figure 5.3 - Architecture logicielle globale de SkyPharma	69
Figure 5.4 - Modèle conceptuel de données de SkyPharma	71
Figure 5.5 - Diagramme de séquence du cycle de commande	72
Figure 5.6 - Diagramme de séquence du cycle de commande	73
Figure 5.7 - Diagramme de séquence du cycle de commande	74
Figure 5.8 - Diagramme de séquence du cycle de commande	75
Figure 5.9 - Diagramme de séquence du cycle de commande	76
Figure 5.10 - Diagramme de séquence du cycle de commande	76
Figure 6.1 - Architecture générale du module de livraison par drone intégré à SkyPharma	79
Figure 6.2 - Châssis F450 utilisé pour la réalisation du drone	80
Figure 6.3 - Contrôleur de vol APM ArduPilot.....	81
Figure 6.4 - Radiocommande FlySky FS-i6 utilisée pour le pilotage manuel	82
Figure 6.5 - Moteur brushless SunnySky 1400 KV.....	83
Figure 6.6 - Hélices de 8 pouces utilisées sur le drone	83
Figure 6.7 - Batterie LiPo 3S 6500 mAh utilisée pour l'alimentation du drone.....	84
Figure 6.8 - Pièce mécanique développée pour le système d'accrochage.....	85
Figure 6.9 - Séquence simplifiée du mécanisme d'accrochage et de largage commandé par servomoteur.....	86
Figure 6.10 - Schéma électronique général du drone	87
Figure 6.11 - Diagramme simplifié de la séquence de livraison par drone.....	88

Liste des tableaux

Tableau 2.1 - Fonctions principales du système	28
Tableau 2.2 - Fonctions contraintes du système	28
Tableau 2.3 - Synthèse des exigences du système	30
Tableau 2.4 - Synthèse des exigences du système	31
Tableau 2.5 - Synthèse des contraintes de conception	31
Tableau 2.6 - Matrice de justification des outils retenus	32
Tableau 3.1 - Synthèse des principaux éléments mécaniques du robot H-Bot.....	39
Tableau 3.2 - Fonctions mécaniques du prototype.....	49
Tableau 3.3 - Modes de fonctionnement ou de commande.....	49
Tableau 3.4 - Affectation des axes et composants	51
Tableau 3.5 - Connexions principales de l'armoire	54
Tableau 4.1 - Réponses GRBL et interprétation.....	60
Tableau 4.2 - Principales commandes GRBL utilisées	63
Tableau 4.3 - Paramètres GRBL de référence.....	63
Tableau 5.1 - Organisation des tables de la base de données	72
Tableau 6.1 - États du mécanisme d'accrochage.....	85
Tableau 6.2 - Commande logique du servomoteur.....	86
Tableau 6.3 - Rôle des principaux composants du drone	87
Tableau 6.4 - Risques et mesures de prévention.....	88
Tableau 6.5 - Plan de validation du drone	89
Tableau 6.6 - Sources et références consultées	92

Liste des abréviations

Abréviation	Signification
ASRS	Automated Storage and Retrieval System
BDD	Base de données
CNC	Computer Numerical Control
DRV8825	Driver de moteur pas à pas
G-code	Langage de commande numérique
GRBL	Firmware libre de contrôle CNC
H-Bot	Architecture cartésienne à courroie fermée
NEMA17	Format de moteur pas à pas
SQLite	Système de gestion de base de données embarquée
UML	Unified Modeling Language

Tableau P.1 - Liste des abréviations utilisées dans le rapport

INTRODUCTION GÉNÉRALE

Le secteur de la santé connaît aujourd'hui une transformation importante sous l'effet du développement numérique, de l'automatisation, de l'Internet des Objets et de l'intelligence artificielle. Cette évolution ne concerne pas uniquement les hôpitaux et les plateformes médicales, mais touche également les pharmacies, la logistique de santé et les services de distribution des **médicaments**.

Au Maroc, cette transformation s'inscrit dans une dynamique nationale plus large. La stratégie Maroc Digital 2030 vise à accélérer la transformation numérique, à promouvoir l'innovation et à renforcer l'inclusion digitale dans différents secteurs, dont les services publics et les domaines à fort impact social. Dans le domaine de la santé, les réformes récentes mettent également l'accent sur la modernisation du système de santé et sur l'implémentation d'un système d'information national unifié afin de faciliter l'échange des données médicales entre les acteurs du secteur.

Dans ce contexte, l'accès rapide et fiable aux **médicaments** devient un enjeu important. Les pharmacies traditionnelles assurent un rôle essentiel dans la distribution des traitements, mais elles peuvent rencontrer plusieurs contraintes : attente des clients, erreurs de préparation, gestion manuelle du **stock**, rupture de certains produits, difficulté de suivi des commandes et surcharge du personnel. Ces limites deviennent plus visibles lorsque la demande augmente ou lorsque les **médicaments** doivent être livrés rapidement.

Parallèlement, les solutions de robotisation pharmaceutique se développent dans plusieurs pays. Des systèmes automatisés comme BD Rowa, Omnicell, Swisslog ou Apostore permettent d'automatiser le **stockage**, la préparation et la distribution des **médicaments**. BD Rowa propose par exemple des robots capables de gérer les niveaux de **stock** en temps réel à travers une interface avec le système de gestion de la pharmacie. Swisslog Healthcare présente également des solutions d'automatisation permettant le **stockage**, la récupération, le conditionnement et la distribution des **médicaments** dans les pharmacies hospitalières.

Cependant, ces systèmes restent généralement coûteux, volumineux et destinés à des structures professionnelles de grande taille. Le projet Sky Pharma cherche donc à proposer une approche pédagogique et expérimentale inspirée de ces solutions industrielles, mais adaptée à une réalisation académique à petite échelle.

La problématique générale de ce projet peut être formulée comme suit :

Comment concevoir et réaliser une plateforme intelligente capable d'automatiser la distribution des **médicaments**, d'améliorer la gestion des commandes et d'offrir plusieurs modes de récupération adaptés aux besoins des utilisateurs ?

Cette problématique principale se divise en deux axes.

Le premier axe concerne la distribution locale des **médicaments**. Dans une pharmacie classique, la préparation d'une commande dépend fortement de l'intervention humaine. Le pharmacien ou l'opérateur doit rechercher le **médicament**, vérifier sa disponibilité, préparer la

INTRODUCTION GÉNÉRALE

commande, puis la remettre au client. Cette organisation peut entraîner un temps d'attente important, des erreurs de sélection, une mauvaise **traçabilité** et une difficulté à suivre l'état du **stock** en temps réel.

Le deuxième axe concerne la livraison des **médicaments**. Certaines situations nécessitent une livraison rapide, notamment lorsqu'un patient se trouve dans une zone éloignée, lorsqu'un **médicament** est demandé en urgence ou lorsque le déplacement du client est difficile. Les services de livraison classiques restent utiles, mais ils peuvent être limités par le trafic, la distance ou la disponibilité des livreurs. Dans le monde, des entreprises comme Zipline ont montré l'intérêt des drones pour la livraison de produits médicaux, avec des drones capables d'effectuer des trajets aller-retour importants et de livrer des colis médicaux dans des zones ciblées.

Ainsi, Sky Pharma propose une solution intégrant une station automatisée de distribution et plusieurs modes de récupération : récupération locale, livraison par partenaire et **livraison par drone** comme extension future ou solution complémentaire.

L'objectif général du projet est de concevoir et réaliser une plateforme intelligente de distribution et de livraison automatisée de **médicaments**.

Les objectifs spécifiques sont les suivants :

- concevoir une architecture mécanique permettant le déplacement automatique d'un système de récupération ;
- réaliser un robot cartésien capable d'accéder à plusieurs emplacements de stockage ;
- piloter les axes à l'aide de moteurs pas à pas, d'une carte Arduino UNO, d'une CNC Shield V3 et de drivers DRV8825 ;
- adapter le firmware GRBL au fonctionnement du système ;
- développer une communication entre l'application logicielle et le contrôleur embarqué ;
- générer automatiquement les commandes G-code nécessaires au déplacement du robot ;
- concevoir une base de données pour la gestion du stock, des commandes et des utilisateurs ;
- développer des interfaces pour le client, l'opérateur et l'administrateur ;
- proposer une architecture de livraison complémentaire par drone ;
- valider expérimentalement le fonctionnement du système réalisé

La méthodologie suivie dans ce projet repose sur une démarche progressive. La première étape consiste à étudier les solutions existantes dans le domaine de l'automatisation pharmaceutique afin d'identifier leurs principes de fonctionnement, leurs avantages et leurs limites. La deuxième étape concerne l'analyse fonctionnelle du besoin à travers des outils tels que la bête à cornes, le diagramme pieuvre, le diagramme FAST et le cahier des charges.

La troisième étape porte sur la conception globale de la plateforme. Elle comprend le choix de l'architecture mécanique, le dimensionnement des axes, le choix des composants électroniques, la définition de l'architecture logicielle et la conception de la **base de données**. La quatrième étape correspond à la réalisation pratique : assemblage mécanique, câblage électronique, configuration du firmware, développement logiciel et création des interfaces.

INTRODUCTION GÉNÉRALE

Enfin, la dernière étape est consacrée aux tests et à la validation. Elle permet de vérifier le bon fonctionnement du robot, la précision des déplacements, la communication entre les différentes parties du système, la gestion des commandes et la cohérence du cycle automatique.

Ce rapport est organisé en six chapitres, suivis d'une conclusion générale et d'annexes.

Le premier chapitre présente l'état de l'art des pharmacies automatisées, des robots ASRS et des solutions de livraison intelligente.

Le deuxième chapitre est consacré à l'analyse fonctionnelle et à la conception globale de Sky Pharma.

Le troisième chapitre regroupe la conception mécanique et électronique du robot ASRS, afin de présenter dans une même partie la structure, l'architecture H-Bot, l'armoire électrique et les validations électromécaniques.

Le quatrième chapitre traite du développement embarqué, de la configuration de GRBL, du paramétrage des axes et de la communication série.

Le cinquième chapitre présente le développement logiciel, la base de données, les interfaces distante et locale, ainsi que les pistes d'amélioration par intelligence artificielle.

Le sixième chapitre décrit le système de livraison par drone, son architecture, ses composants, son mécanisme de largage, sa sécurité et sa validation.

Chapitre 1 : État de l'art et étude des solutions existantes

1.1 Introduction

L'automatisation des pharmacies représente aujourd'hui une évolution importante dans le domaine de la distribution des **médicaments**. Elle vise à améliorer la rapidité de service, la sécurité de préparation, la gestion du **stock** et la **traçabilité** des opérations. Dans les environnements hospitaliers ou les pharmacies à forte activité, l'automatisation permet aussi de réduire la charge de travail du personnel et de limiter les erreurs liées à la manipulation manuelle des **médicaments**.

Ce chapitre présente d'abord le fonctionnement d'une pharmacie traditionnelle, puis les limites de ce fonctionnement. Ensuite, il introduit les systèmes automatisés de **stockage** et de récupération utilisés dans le domaine pharmaceutique. Enfin, une étude comparative permet d'identifier les besoins non couverts et de justifier la contribution proposée par Sky Pharma.

1.2 Problématique d'accès aux médicaments dans les zones isolées et en situation d'urgence

L'accès aux **médicaments** constitue un enjeu majeur de santé publique, particulièrement dans les zones rurales et isolées où les infrastructures médicales sont limitées. Dans ces régions, la distance entre les patients et les pharmacies ou centres de santé peut atteindre plusieurs dizaines de kilomètres, ce qui entraîne des retards critiques dans la prise en charge des patients. Selon l'Organisation Mondiale de la Santé, près de 50 % de la population mondiale n'a pas un accès régulier aux **médicaments** essentiels, une situation encore plus marquée dans les zones rurales des pays en développement.

Les situations d'urgence illustrent fortement cette problématique. Par exemple, les envenimations par piqûres de scorpions représentent un problème de santé important dans plusieurs régions, notamment en Afrique du Nord. Au Maroc, on enregistre chaque année plus de 25 000 cas de piqûres de scorpions, dont plusieurs centaines nécessitent une intervention urgente. Le délai d'accès au sérum antivenimeux est un facteur déterminant dans la survie du patient, et tout retard peut entraîner des complications graves, voire mortelles.

De même, la disponibilité rapide des produits sanguins constitue un défi critique. Dans les situations d'hémorragie sévère, chaque minute compte. Pourtant, dans certaines zones éloignées, l'acheminement des poches de sang peut prendre plusieurs heures en raison des contraintes logistiques et du manque de moyens de transport adaptés. Selon certaines études, les retards dans la transfusion sanguine sont responsables d'une part significative des décès évitables dans les zones rurales.

Ces constats mettent en évidence la nécessité de développer des solutions innovantes capables de réduire les délais d'accès aux **médicaments** et aux produits médicaux essentiels. Le projet Sky Pharma s'inscrit dans cette logique en proposant une plateforme intelligente

combinant automatisation et solutions de livraison adaptées, afin de répondre efficacement aux besoins des populations, notamment dans les situations critiques et les zones difficiles d'accès.



Figure 1.1 - Contexte d'une officine et limites de distribution traditionnelle

1.3 Robotisation des pharmacies

La robotisation des pharmacies consiste à utiliser des systèmes automatisés capables de **stocker**, rechercher et distribuer les **médicaments** avec un minimum d'intervention humaine. Ces systèmes sont généralement composés d'une zone de **stockage**, d'un mécanisme de déplacement, d'un système de préhension, de capteurs, d'un logiciel de gestion et d'une interface avec le système informatique de la pharmacie.

Le principe général est le suivant : lorsqu'une commande est validée, le logiciel identifie l'emplacement du **médicament** dans le **stock**. Le robot se déplace ensuite vers cet emplacement, récupère le produit demandé, puis l'achemine vers une zone de sortie. Ce processus permet d'améliorer la rapidité de préparation, la **traçabilité**, la gestion du **stock** et la sécurité de distribution.

Les robots de pharmacie peuvent adopter plusieurs architectures : **robot cartésien**, bras robotisé, système de convoyage, **stockage** vertical, carrousel automatisé ou système **ASRS**. Le choix dépend du volume de **stockage**, de la vitesse souhaitée, du budget, de l'espace disponible et du type de produits à manipuler.



Figure 1.2 - Références visuelles de systèmes automatisés de pharmacie

1.4 Robots ASRS industriels

Les systèmes **ASRS**, ou Automated Storage and Retrieval Systems, regroupent plusieurs types d'architectures utilisées pour automatiser le **stockage** et la récupération des produits. Parmi les principaux types, on distingue :

Les systèmes à trans**stockeur** (stacker crane), utilisés dans les entrepôts à grande hauteur, permettant de déplacer des charges lourdes sur plusieurs niveaux.

Les systèmes à carrousel horizontal ou vertical, où les produits tournent autour d'un axe pour être présentés à l'opérateur.

Les systèmes à navettes (shuttle systems), qui utilisent des robots mobiles pour transporter les produits entre les rayonnages.

Les systèmes à **robot cartésien**, basés sur des axes X, Y et parfois Z, permettant un déplacement précis dans un espace défini.

Les systèmes à bras robotisé, capables de manipuler des objets avec une grande flexibilité.

Dans le domaine pharmaceutique, ces systèmes permettent de **stocker** un grand nombre de boîtes de **médicaments** dans un espace optimisé et de les récupérer automatiquement à la demande. Des solutions industrielles comme BD Rowa, Omnicell, Swisslog ou Apostore illustrent ces technologies avancées, avec des systèmes capables d'assurer le **stockage**, la distribution, la **traçabilité** et la gestion intelligente des **médicaments**.

Cependant, chaque type d'**ASRS** présente des limitations. Les systèmes à trans**stockeur** sont coûteux et nécessitent un espace important. Les carrousels peuvent être limités en vitesse et en capacité d'accès simultané. Les systèmes à navettes sont complexes à gérer et demandent une maintenance avancée. Les bras robotisés offrent une grande flexibilité mais sont souvent chers et difficiles à programmer. Enfin, les solutions industrielles globales restent généralement lourdes, coûteuses et peu adaptées à des projets académiques ou à petite échelle.

Dans ce contexte, le choix de l'architecture du projet Sky Pharma s'est orienté vers un **robot cartésien** de type **H-Bot**. Cette architecture présente plusieurs avantages importants. Elle permet un déplacement rapide et précis sur deux axes avec un nombre réduit de moteurs, ce qui simplifie la conception mécanique et électronique. Le système **H-Bot** utilise une seule courroie pour piloter les axes X et Y, ce qui réduit le coût, le poids et la complexité du montage.

De plus, l'architecture **H-Bot** est particulièrement adaptée aux applications nécessitant une bonne précision de positionnement dans un espace plan, comme le **stockage** et la récupération de **médicaments**. Elle offre une structure compacte, facile à intégrer dans une station de petite taille, tout en garantissant une bonne répétabilité des mouvements.

Ainsi, contrairement aux solutions industrielles complexes, le choix du **H-Bot** dans Sky Pharma permet de proposer une solution optimisée, économique et adaptée à un environnement académique, tout en conservant les principes fondamentaux des systèmes **ASRS** modernes.

1.5 Pharmacies intelligentes

Une pharmacie intelligente ne se limite pas à l'utilisation d'un robot. Elle repose sur l'intégration de plusieurs technologies : gestion numérique du **stock**, interface utilisateur, **traçabilité**, communication réseau, **base de données**, supervision et éventuellement intelligence artificielle.

Dans une telle architecture, le client peut passer une commande à travers une interface. Le système vérifie automatiquement la disponibilité du **médicament**, enregistre la commande, choisit le mode de récupération puis transmet les informations au système de distribution. L'opérateur peut suivre l'état des commandes, contrôler le robot, gérer les alarmes et mettre à jour le **stock**. L'administrateur peut gérer les utilisateurs, consulter les statistiques et configurer la plateforme.

Cette approche permet de passer d'une pharmacie centrée uniquement sur la manipulation manuelle vers une pharmacie connectée, traçable et partiellement automatisée.

1.6 Livraison intelligente des médicaments

La livraison intelligente des **médicaments** constitue une extension importante des pharmacies automatisées. Elle peut prendre plusieurs formes : livraison locale par un agent, livraison par un partenaire logistique, casier automatique, robot mobile ou drone.

La **livraison par drone** présente un intérêt particulier pour les situations d'urgence ou les zones difficiles d'accès. Elle peut réduire le temps de livraison lorsque les routes sont longues, encombrées ou difficiles à emprunter. Des entreprises comme Zipline utilisent déjà des drones pour transporter des produits médicaux dans certains pays, avec un modèle basé sur des hubs de préparation et des drones autonomes capables de livrer rapidement les commandes.

Dans le cadre du projet Sky Pharma, la **livraison par drone** est considérée comme une extension complémentaire. Elle ne remplace pas la pharmacie ni le rôle du pharmacien, mais elle propose une solution supplémentaire pour certains cas particuliers : urgence, distance, difficulté d'accès ou besoin de rapidité.

1.7 Technologies utilisées dans Sky Pharma

Le projet Sky Pharma repose sur l'intégration de plusieurs technologies complémentaires couvrant les domaines mécanique, électronique et logiciel, afin de réaliser une plateforme intelligente de distribution automatisée de **médicaments**.

Sur le plan mécanique, le système est basé sur un **robot cartésien** utilisant une architecture de type **H-Bot**. Cette configuration permet de déplacer un chariot sur deux axes (X et Y) avec précision et stabilité. Le guidage est assuré par des rails linéaires, tandis que la transmission du mouvement est réalisée à l'aide de courroies et de poulies. Cette solution offre un bon compromis entre simplicité de conception, coût réduit et précision de positionnement. Un système de récupération est intégré pour saisir et transporter les **médicaments** depuis leur emplacement de **stockage** vers la zone de sortie.

Sur le plan électronique, la commande du système est assurée par une carte **Arduino UNO** associée à une **CNC Shield V3**. Les moteurs pas à pas sont pilotés à l'aide de drivers **DRV8825**, permettant un contrôle précis des déplacements. Des capteurs de **fin de course** sont utilisés pour définir les positions de référence et assurer la sécurité du système. L'ensemble est alimenté par une source adaptée garantissant la stabilité du fonctionnement.

Sur le plan logiciel, le système repose sur une architecture client/serveur. Une **base de données** est utilisée pour gérer les informations relatives aux **médicaments**, aux utilisateurs et aux commandes. Une application permet aux utilisateurs de passer des commandes, tandis qu'une interface opérateur assure la supervision du système. La communication entre le logiciel et le contrôleur embarqué se fait via une liaison série, avec l'envoi de commandes **G-code** générées automatiquement pour piloter les déplacements du robot.

Enfin, le projet intègre également une dimension évolutive avec la possibilité d'ajouter des fonctionnalités liées à l'Internet des Objets pour la supervision à distance, ainsi qu'une solution complémentaire de **livraison par drone** pour répondre à des besoins spécifiques tels que les situations d'urgence ou les zones difficiles d'accès.

1.8 Conclusion

L'étude de l'existant montre que l'automatisation pharmaceutique constitue aujourd'hui une évolution majeure dans le domaine de la santé. Les solutions industrielles analysées permettent d'améliorer la gestion des **médicaments**, d'optimiser les flux et de réduire les erreurs humaines. Toutefois, ces systèmes restent généralement coûteux, complexes à mettre en œuvre et peu accessibles dans un cadre académique ou pour des structures de petite taille.

Dans ce contexte, le projet Sky Pharma s'inscrit comme une approche simplifiée et adaptée, visant à reproduire les principes fondamentaux de ces solutions à travers une architecture modulaire et accessible. Il propose la conception d'une station intelligente capable d'assurer le **stockage**, la récupération et la distribution automatisée des **médicaments**, tout en intégrant une gestion logicielle et plusieurs modes de récupération.

Cependant, plusieurs questions restent ouvertes et nécessitent une étude approfondie : comment traduire ces besoins en fonctions techniques concrètes ? Quelle architecture globale

Chapitre 1 : État de l’art et étude des solutions existantes

adopter pour assurer la cohérence entre les parties mécanique, électronique et logicielle ? Et comment garantir un fonctionnement fiable et efficace du système proposé ?

Ces interrogations constituent le point de départ du chapitre suivant, consacré à l’analyse fonctionnelle et à la conception globale de la plateforme Sky Pharma.

Chapitre 2 : Analyse fonctionnelle et conception globale

2.1 Introduction

Après avoir présenté l'état de l'art des solutions existantes dans le domaine de l'automatisation pharmaceutique, ce chapitre est consacré à l'analyse fonctionnelle et à la conception globale de la plateforme Sky Pharma.

L'objectif de cette partie est de passer d'une idée générale, à savoir automatiser la distribution et la livraison des **médicaments**, vers une architecture concrète, structurée et réalisable. Cette étape est essentielle dans un projet mécatronique, car elle permet de définir clairement le besoin, les acteurs concernés, les fonctions attendues, les contraintes à respecter et les choix d'architecture avant de passer à la conception mécanique, électronique et logicielle.

Le chapitre commence par une présentation générale de Sky Pharma et de son principe de fonctionnement. Ensuite, une analyse du besoin est réalisée à travers la bête à cornes, le diagramme pieuvre, le diagramme FAST et le diagramme SADT. Un cahier des charges fonctionnel est ensuite établi afin de traduire les besoins en exigences techniques. Enfin, l'architecture globale du système est présentée, suivie des principaux diagrammes UML permettant de modéliser les interactions entre les utilisateurs, le logiciel, la **base de données** et le robot.

2.2 Présentation générale de Sky Pharma

2.2.1 Vision du projet

Sky Pharma est une plateforme intelligente destinée à automatiser la distribution et la livraison de **médicaments**. Elle combine un système de **stockage** automatisé, un robot de récupération, une **base de données**, une interface utilisateur et plusieurs modes de récupération.

L'idée principale est de permettre à un client de rechercher un **médicament**, de vérifier sa disponibilité, de passer une commande, puis de choisir le mode de récupération le plus adapté. Une fois la commande validée, le système identifie l'emplacement du **médicament** dans le **stock**, génère automatiquement les instructions de déplacement du robot et prépare le produit demandé.

Sky Pharma ne vise pas à remplacer le pharmacien. Au contraire, le système est conçu comme un outil d'assistance permettant de réduire les tâches répétitives, d'améliorer la **traçabilité**, de limiter les erreurs humaines et d'accélérer la préparation des commandes.

2.2.2 Principe général

1. Le fonctionnement général de Sky Pharma repose sur l'enchaînement suivant :
 - Le client accède à l'interface de commande.
 - Il recherche le médicament souhaité.
 - Le système vérifie la disponibilité du médicament dans la base de données.

- Le client choisit le mode de récupération.
- La commande est enregistrée.
- Le système détermine l'emplacement du médicament.
- Le module logiciel génère le G-code correspondant au mouvement du robot.
- Le contrôleur embarqué exécute les déplacements nécessaires.
- Le médicament est récupéré puis déposé dans une zone de sortie.
- Le stock et l'historique de la commande sont mis à jour.

Ce principe permet de relier le monde numérique, représenté par l'interface et la **base de données**, au monde physique, représenté par le robot, les moteurs, les capteurs et le plateau de récupération.

Fonctionnement global de SkyPharma

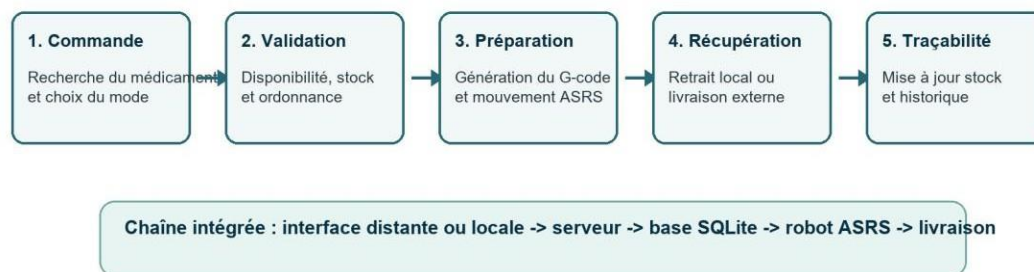


Figure 2.1 - Fonctionnement global de SkyPharma

Acteurs du système

Le système Sky Pharma met en interaction plusieurs acteurs.

- Client ou patient

Le client est l'utilisateur final du service. Il peut rechercher un **médicament**, consulter sa disponibilité, passer une commande, choisir un mode de récupération et suivre l'état de sa commande.

Pharmacien ou opérateur

L'opérateur supervise le fonctionnement du système. Il peut valider certaines commandes, lancer le **homing** du robot, contrôler l'état de la machine, intervenir en cas d'erreur, remplir les emplacements de **stockage** et gérer les alarmes.

- Administrateur

L'administrateur gère les paramètres globaux du système. Il peut créer ou modifier les comptes utilisateurs, gérer les droits d'accès, consulter les statistiques, configurer les emplacements et suivre l'historique des opérations.

- Partenaire de livraison

Le partenaire de livraison intervient lorsque le client choisit le mode de livraison externe. Il récupère le **médicament** préparé par la station et assure l'acheminement vers le client.

- Drone de livraison

Le drone est considéré comme une extension complémentaire du système. Il peut être utilisé dans des cas particuliers, notamment pour des livraisons urgentes, des zones éloignées ou des environnements difficiles d'accès. Dans le cadre du présent projet, cette partie est présentée comme une architecture évolutive pouvant être développée et validée dans une phase avancée.

2.3 Modes de distribution et de livraison

Sky Pharma intègre plusieurs modes de distribution afin d'adapter le service aux différents contextes d'utilisation et aux besoins des utilisateurs.

Mode 1 : distribution locale automatisée

Dans ce mode, le système assure la préparation automatique du **médicament** au niveau de la station Sky Pharma. Le robot identifie l'emplacement du produit, effectue les déplacements nécessaires et dépose le **médicament** dans une zone de sortie sécurisée.

Ce mode est particulièrement adapté aux environnements tels que les pharmacies, les hôpitaux, les campus universitaires ou les stations intelligentes en milieu urbain.

Mode 2 : distribution avec livraison par partenaire

Dans ce mode, après la préparation automatique du **médicament** par le système, un partenaire logistique prend en charge l'acheminement du produit vers une adresse définie.

Ce mode permet d'étendre le service au-delà de la station physique tout en conservant une préparation automatisée, fiable et traçable.

Mode 3 : distribution avec livraison par drone

Ce mode correspond à une solution complémentaire basée sur l'utilisation de drones pour le transport des **médicaments**. Il est principalement destiné aux situations d'urgence ou aux zones difficiles d'accès.

Le système prépare le **médicament**, l'associe à une mission de livraison et transmet les informations nécessaires au module drone. Le drone assure ensuite le transport du colis vers une zone cible définie.

Ce mode nécessite des conditions spécifiques en termes de sécurité, de réglementation et d'autorisation. Il est donc considéré comme une extension évolutive du système, intégrée dès la phase de conception globale.

2.4 Analyse du besoin

2.4.1 Expression du besoin

Le besoin principal peut être exprimé de la manière suivante :

Permettre à un client d'obtenir rapidement un **médicament** disponible, à travers une station intelligente capable de gérer la commande, de récupérer automatiquement le produit et de proposer plusieurs modes de récupération.

Ce besoin ne se limite pas uniquement à la rapidité d'accès au **médicament**, mais englobe également la fiabilité du système, la réduction des erreurs humaines et l'amélioration de l'expérience utilisateur. En effet, dans un contexte pharmaceutique, il est essentiel de garantir que le bon **médicament** est délivré au bon moment, tout en assurant une **traçabilité** complète des opérations.

Ce besoin met en évidence trois dimensions importantes :

une dimension physique : elle concerne l'ensemble des éléments matériels du système, tels que le **stockage** organisé des **médicaments**, le déplacement du robot dans l'espace, la précision de la récupération et le dépôt sécurisé du produit dans la zone de sortie. Cette dimension implique également des contraintes mécaniques comme la stabilité, la précision et la répétabilité des mouvements ;

une dimension numérique : elle regroupe toutes les fonctions liées au traitement de l'information, notamment la gestion des commandes, l'interaction avec l'utilisateur via une interface, la communication avec la **base de données** et la **traçabilité** des opérations. Cette dimension permet d'assurer la cohérence des données, la mise à jour du **stock** en temps réel et la gestion des historiques ;

une dimension logistique : elle concerne les différentes manières dont le **médicament** peut être remis au client. Cela inclut la récupération locale directement à la station, la livraison via un partenaire externe ou encore la **livraison par drone**. Cette dimension permet d'adapter le service aux besoins spécifiques des utilisateurs et d'étendre la portée du système au-delà de son emplacement physique.

Ainsi, le besoin global de Sky Pharma s'inscrit dans une approche intégrée combinant automatisation, gestion intelligente des données et flexibilité logistique, afin d'offrir un service efficace, sécurisé et évolutif.

2.4.2 Bête à cornes

La bête à cornes permet de formuler le besoin fondamental du système en répondant à trois questions : à qui le système rend-il service, sur quoi agit-il et dans quel but ?

Bête à cornes - expression du besoin



Figure 2.2 - Bête à cornes du système SkyPharma

La bête à cornes permet de formuler le besoin fondamental du système en répondant à trois questions : à qui le système rend-il service, sur quoi agit-il et dans quel but ?

Le système rend service principalement au client, au pharmacien et à l'opérateur. Il agit sur les commandes, les **médicaments**, le **stock** et les flux de récupération. Son but est d'améliorer l'accès aux **médicaments**, de réduire le temps d'attente, d'assurer une meilleure **traçabilité** et de limiter les erreurs de préparation.

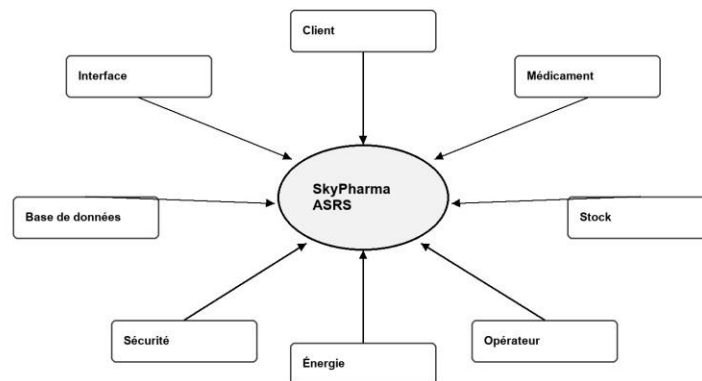
2.5 Outils d'analyse fonctionnelle

Le diagramme pieuvre permet d'identifier les interactions entre Sky Pharma et son environnement extérieur. Il distingue les fonctions principales, qui traduisent le service attendu, et les fonctions contraintes, qui représentent les exigences à respecter.

Figure 2.3 - Diagramme pieuvre de SkyPharma

2.5.1 Fonctions principales

Diagramme pieuvre - interactions de SkyPharma



Code	Fonction principale	Description
FP1	Distribuer un médicament	Préparer automatiquement un produit demandé.
FP2	Gérer le stock	Suivre les quantités disponibles.
FP3	Proposer plusieurs modes	Retrait local, partenaire ou drone.
FP4	Assurer la traçabilité	Enregistrer commandes, mouvements et états.

Tableau 2.1 - Fonctions principales du système

2.5.2 Fonctions contraintes

Code	Contrainte	Description
FC1	Sécurité	Éviter les mouvements dangereux.
FC2	Hygiène	Préserver l'intégrité des médicaments.
FC3	Précision	Atteindre l'emplacement cible.

Code	Contrainte	Description
FC4	Alimentation	Garantir une énergie stable.
FC5	Intervention humaine	Permettre l'arrêt et la correction.
FC6	Évolutivité	Prévoir de nouvelles références.
FC7	Coût raisonnable	Utiliser des composants accessibles.

Tableau 2.2 - Fonctions contraintes du système

2.5.3 Diagramme FAST

le diagramme FAST permet de décomposer la fonction principale du système en sous-fonctions techniques. Il répond à la logique : pourquoi ? comment ?

La fonction principale est : distribuer automatiquement un **médicament**.

Pour atteindre cette fonction, le système doit :

- identifier la commande ;
- vérifier la disponibilité du médicament ;
- localiser son emplacement ;
- générer le mouvement du robot ;
- déplacer le plateau de récupération ;
- prendre le médicament ;
- le déposer dans la zone de sortie ;
- mettre à jour le stock et l'historique.

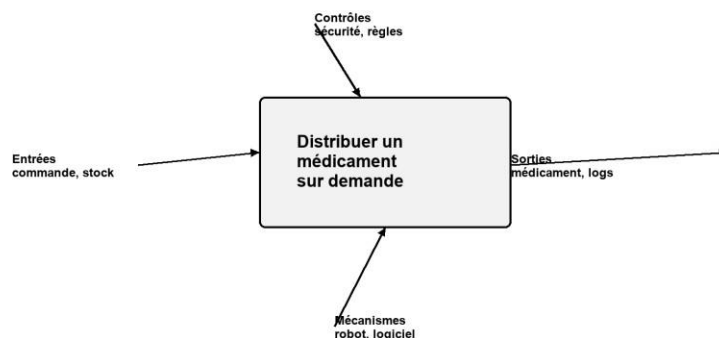
Figure 2.4 - Diagramme FAST de SkyPharma

2.5.4 Diagramme SADT

Le diagramme SADT permet de représenter la fonction globale du système sous forme d'entrées, sorties, contrôles et mécanismes.

Figure 2.5 - Diagramme SADT A-0 de SkyPharma

Diagramme SADT A-0 - fonction globale



Entrées

Les entrées du système sont :

- la demande du client ;
- l'ordonnance ou la référence du médicament ;
- les données utilisateur ;
- l'état du stock ;
- les informations d'emplacement des médicaments.

Sorties

Les sorties attendues sont :

- un médicament préparé ;
- une commande validée ;
- un stock mis à jour ;
- un historique enregistré ;
- une notification de fin de préparation.

Contrôles

Les contrôles correspondent aux règles qui conditionnent le fonctionnement :

- disponibilité du médicament ;
- validation de la commande ;
- règles de sécurité ;
- choix du mode de récupération ;
- limites mécaniques du robot ;
- autorisations utilisateur.

Mécanismes

Les mécanismes utilisés sont :

- le robot H-Bot ;
- l'axe Z et le plateau de récupération ;
- la carte Arduino UNO ;
- la CNC Shield V3 ;
- les drivers DRV8825 ;
- le firmware GRBL ;
- l'application logicielle ;
- la base de données ;
- l'opérateur ou le système de livraison.

2.6 Cahier des charges fonctionnel

Le cahier des charges fonctionnel traduit les besoins en exigences mesurables ou vérifiables.

2.6.1 Exigences fonctionnelles

Référence	Exigence	Critère d'acceptation
EF1	Le client doit pouvoir rechercher un médicament	Recherche par nom ou référence
EF2	Le système doit vérifier la disponibilité	Retour disponible / indisponible
EF3	Le système doit enregistrer une commande	Commande stockée dans la base de données
EF4	Le système doit associer chaque médicament à un emplacement	Coordonnées X, Y, Z disponibles
EF5	Le robot doit récupérer automatiquement le médicament	Cycle de préparation exécuté
EF6	Le système doit proposer trois modes de récupération	Local, partenaire, drone
EF7	Le stock doit être mis à jour après chaque commande	Quantité décrémentée automatiquement
EF8	L'opérateur doit pouvoir lancer le homing	Origine machine correctement définie
EF9	Le système doit enregistrer les événements	Journalisation des actions et erreurs

Tableau 2.3 - Synthèse des exigences du système

2.6.2 Exigences techniques

Référence	Exigence	Solution retenue
ET1	Déplacement sur deux axes horizontaux	Architecture H-Bot
ET2	Mouvement vertical de récupération	Axe Z motorisé
ET3	Commande des moteurs pas à pas	Arduino UNO + CNC Shield V3
ET4	Pilotage moteur	Drivers DRV8825
ET5	Génération des trajectoires	G-code généré automatiquement
ET6	Exécution du mouvement	Firmware GRBL
ET7	Communication logiciel-machine	Liaison série USB
ET8	Gestion des données	Base de données relationnelle
ET9	Supervision	Interface opérateur

Tableau 2.4 - Synthèse des exigences du système

2.6.3 Contraintes

Type de contrainte	Description
Contrainte mécanique	La structure doit être rigide, précise et compatible avec les dimensions des médicaments.

Type de contrainte	Description
Contrainte électronique	Les drivers doivent être correctement refroidis et alimentés.
Contrainte logicielle	Le système doit éviter les commandes incohérentes et gérer les erreurs.
Contrainte de sécurité	Le robot ne doit pas sortir de sa zone de fonctionnement.
Contrainte économique	Le prototype doit utiliser des composants accessibles.
Contrainte d'évolutivité	L'architecture doit permettre l'ajout d'un module drone ou IoT.

Tableau 2.5 - Synthèse des contraintes de conception

2.7 Architecture globale du système

Figure 2.6 - Cartographie technologique du prototype SkyPharma

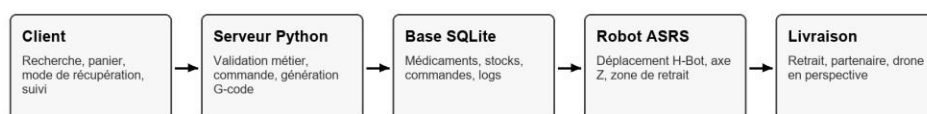
Couche	Outil	Rôle dans SkyPharma	Justification
CAO	SolidWorks	Modélisation mécanique	Assemblage clair et vérification des interférences
Firmware	GRBL	Interprétation G-code	Solution stable pour Arduino et moteurs pas à pas
Contrôle	Arduino UNO + CNC Shield	Commande STEP/DIR	Composants accessibles et documentés
Logiciel	Python / Streamlit	Interfaces et supervision	Développement rapide et lisible
Données	SQLite	Stock et traçabilité	Base légère, locale et suffisante pour prototype

Tableau 2.6 - Matrice de justification des outils retenus

L'architecture globale de Sky Pharma est organisée en plusieurs couches : couche utilisateur, couche serveur, couche **base de données**, couche communication, couche commande embarquée et couche mécanique

Figure 2.7 - Architecture globale du système SkyPharma

Architecture globale du système SkyPharma



Couche utilisateur

- La couche utilisateur regroupe les interfaces client, opérateur et administrateur.
- L'interface client permet de rechercher un médicament, passer une commande, choisir le mode de récupération et suivre l'état de la commande.
- L'interface opérateur permet de superviser le robot, lancer le homing, commander une distribution, consulter les alarmes et gérer le stock.
- L'interface administrateur permet de gérer les utilisateurs, les paramètres du système et les statistiques.

Couche serveur

- Le serveur constitue le cœur logique du système. Il reçoit les requêtes des interfaces, vérifie les données, communique avec la base de données, valide les commandes et transmet les informations nécessaires au module de génération du G-code.

Il joue donc le rôle d'intermédiaire entre l'utilisateur et le système physique.

Couche base de données

La **base de données** assure la gestion des informations suivantes :

- utilisateurs ;
- médicaments ;
- emplacements ;
- commandes ;
- modes de récupération ;
- livraisons ;
- historiques ;
- paramètres machine.

Elle permet d'assurer la cohérence du système, la **traçabilité** des opérations et la mise à jour du **stock**.

Modèle conceptuel simplifié de la base de données

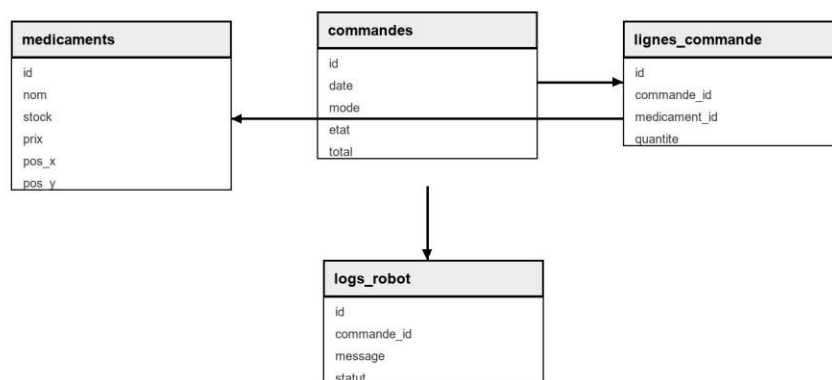


Figure 2.8 - Modèle conceptuel de données de SkyPharma

Couche commande embarquée

- La commande embarquée repose sur une carte Arduino UNO équipée du firmware GRBL. Elle reçoit les commandes G-code via une liaison série et génère les signaux nécessaires au pilotage des moteurs.
- La CNC Shield V3 facilite la connexion entre l'Arduino, les drivers DRV8825, les moteurs pas à pas et les fins de course.

Couche mécanique

- La partie mécanique est constituée d'un robot cartésien de type H-Bot, d'un axe Z, d'un plateau de récupération, de courroies, de poulies, de guidages linéaires et d'une structure de stockage.

Cette architecture permet de déplacer le plateau vers un emplacement donné, de récupérer le **médicament** et de le déposer dans une zone de sortie.

2.8 Architecture des flux

Flux d'information

- Le flux d'information commence par l'utilisateur. Une commande est transmise au serveur, puis enregistrée dans la base de données. Le serveur récupère les coordonnées de l'emplacement du médicament, puis transmet les données au module logiciel chargé de générer le G-code. Les instructions sont ensuite envoyées à l'Arduino via la liaison série.
- Le retour d'état du robot permet de confirmer la fin du cycle ou de signaler une erreur.

Flux physique

- Le flux physique concerne le déplacement réel du médicament. Le médicament est stocké dans un emplacement identifié. Le robot se déplace jusqu'à cet emplacement, active le système de récupération, transporte le médicament, puis le dépose dans la zone de sortie.

Flux de décision

- Le flux de décision intervient principalement lors de la vérification de la disponibilité, du choix du mode de récupération, de la validation de la commande et de la gestion des erreurs.

2.9 Diagrammes UML

2.9.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présente les interactions principales entre les acteurs et le système.

Diagramme de cas d'utilisation

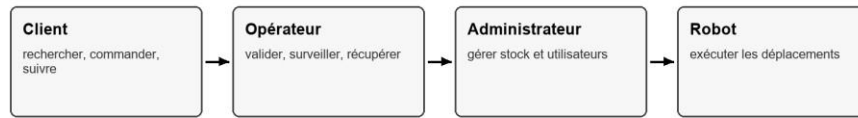


Figure 2.9 - Diagramme de cas d'utilisation de SkyPharma

Les principaux cas d'utilisation sont :

- s'authentifier ;
- rechercher un médicament ;
- passer une commande ;
- choisir un mode de récupération ;
- suivre la commande ;
- lancer le homing ;
- distribuer un médicament ;
- gérer le stock ;
- gérer les utilisateurs ;
- consulter les statistiques ;
- livrer une commande.

2.9.2 Diagramme de séquence

Le diagramme de séquence décrit l'enchaînement chronologique des échanges lors d'une commande locale automatisée.

Figure 2.10 - Diagramme de séquence du cycle de commande

Séquence de traitement d'une commande



- Le système passe par les états suivants :
 1. attente de commande ;
 2. vérification du stock ;
 3. commande validée ;

4. homing si nécessaire ;
5. déplacement vers l'emplacement ;
6. prise du médicament ;
7. dépôt en sortie ;
5. mise à jour du stock ;
9. commande terminée.

En cas d'erreur, comme une indisponibilité du **médicament**, un blocage mécanique ou un défaut de **fin de course**, le système passe vers un état d'alarme nécessitant une intervention ou une réinitialisation.

2.10 Gestion des erreurs, sécurité et justification

La sécurité est un élément essentiel dans un système automatisé de distribution. Même si le prototype est réalisé à petite échelle, il doit intégrer une logique de prévention des défauts.

Les principales erreurs possibles sont :

- médicament indisponible ;
- mauvais emplacement ;
- perte de référence machine ;
- fin de course non détectée ;
- blocage mécanique ;
- erreur de communication série ;
- surchauffe des drivers ;
- coupure d'alimentation ;
- commande incohérente.

Pour limiter ces risques, plusieurs solutions sont prévues :

- homing obligatoire avant le cycle automatique ;
- activation des limites logicielles et matérielles ;
- vérification de la disponibilité avant validation ;
- journalisation des erreurs ;
- arrêt du cycle en cas de défaut ;
- supervision opérateur ;
- refroidissement des drivers ;
- séparation entre commande utilisateur et commande machine.

Justification des choix d'architecture

- Choix d'une architecture H-Bot

Le choix du **H-Bot** est justifié par sa simplicité, son faible coût et sa capacité à assurer un déplacement rapide dans un plan X-Y. Cette architecture est adaptée à une station de **stockage** où le robot doit atteindre plusieurs emplacements répartis dans une grille.

Elle permet également de limiter le nombre de moteurs nécessaires pour le déplacement horizontal, ce qui réduit la complexité électronique et mécanique.

- Choix d'Arduino UNO et GRBL

Arduino UNO associé à **GRBL** constitue une solution simple, connue et largement utilisée pour le pilotage de machines à commande numérique. **GRBL** permet d'interpréter des commandes **G-code** et de piloter les moteurs pas à pas avec précision.

Ce choix facilite le développement du prototype, car il évite de programmer entièrement la génération des impulsions moteurs. Le travail se concentre plutôt sur l'adaptation des paramètres, la génération automatique du **G-code** et l'intégration logicielle.

- Choix d'une architecture client/serveur

L'architecture client/serveur permet de séparer l'interface utilisateur, la logique métier, la **base de données** et le contrôle machine. Cette séparation rend le système plus évolutif et plus facile à maintenir.

Elle permet aussi d'ajouter plus tard des fonctionnalités comme la supervision distante, les notifications, les statistiques, l'IoT ou l'intégration d'un module drone.

- Choix d'une base de données relationnelle

La **base de données** relationnelle est adaptée à la gestion des **médicaments**, commandes, utilisateurs et emplacements. Elle permet d'assurer la cohérence des données, la **traçabilité** des commandes et la mise à jour du **stock**.

Chapitre 3 : Conception mécanique et électronique du robot ASRS

3.1 Introduction

Ce chapitre présente la conception mécanique du robot ASRS SkyPharma à partir des vues SolidWorks et des photos de conception du prototype. L'objectif est d'expliquer clairement l'architecture retenue, les éléments de guidage, les pièces de fixation et les points de validation mécanique.

Le mécanisme retenu est une architecture de type H-Bot. Cette solution permet de déplacer un chariot mobile dans le plan de travail en utilisant une transmission par courroie et une structure légère. Le choix est cohérent avec un prototype académique, car il limite la masse embarquée, facilite l'assemblage et permet d'étudier les principes de guidage, de transmission et de positionnement.

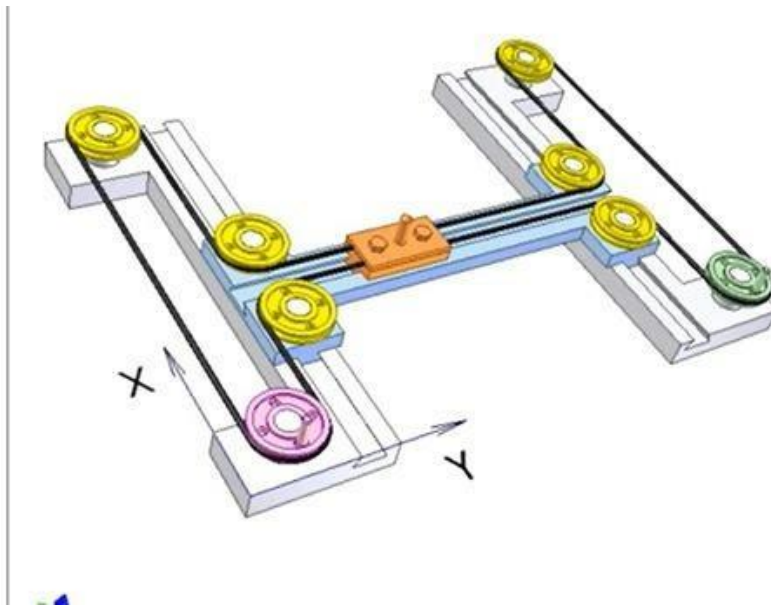


Figure 3.1 - Principe général de l'architecture H-Bot retenue pour le robot ASRS

3.2 Architecture mécanique générale

L'ensemble mécanique est organisé autour de trois fonctions principales : supporter la structure, guider le mouvement et transmettre l'effort. La structure porteuse assure la rigidité globale ; les éléments de guidage imposent la trajectoire du chariot ; la transmission H-Bot synchronise les déplacements afin de placer le module de récupération devant l'emplacement demandé.

L'architecture finale est volontairement modulaire. Les profilés, tiges de guidage, roulements et supports peuvent être démontés ou remplacés sans modifier tout le prototype. Cette approche facilite les essais, le réglage des jeux mécaniques et les corrections de conception après validation.

Tableau 3.1 - Synthèse des principaux éléments mécaniques du robot H-Bot

Élément	Rôle mécanique	Choix retenu
Châssis porteur	Supporter l'ensemble du mécanisme et maintenir l'équerrage.	Profilés aluminium assemblés en structure rigide.
Guidage vertical	Assurer le déplacement régulier du chariot sur l'axe vertical.	Profilé V-Slot, roues de guidage et platine de chariot.
Guidage transversal	Maintenir la trajectoire et limiter les jeux latéraux.	Tiges cylindriques avec supports et roulements linéaires.
Transmission H-Bot	Transformer la rotation des moteurs en déplacement du chariot.	Architecture H-Bot à courroie, compacte et adaptée au prototype.
Pièces de fixation	Positionner les tiges, les roulements et les sous-ensembles.	Supports modélisés sous SolidWorks et pièces dédiées.
Module de récupération	Recevoir le médicament et l'amener vers la zone de sortie.	Chariot mobile intégré au système de guidage.

3.3 Structure porteuse

Le châssis constitue la base du système. Il doit rester stable pendant les phases de déplacement, maintenir l'alignement des axes et supporter les efforts transmis par les courroies, les tiges et le chariot. La structure a été modélisée sous SolidWorks afin de vérifier l'encombrement général et la position relative des sous-ensembles.

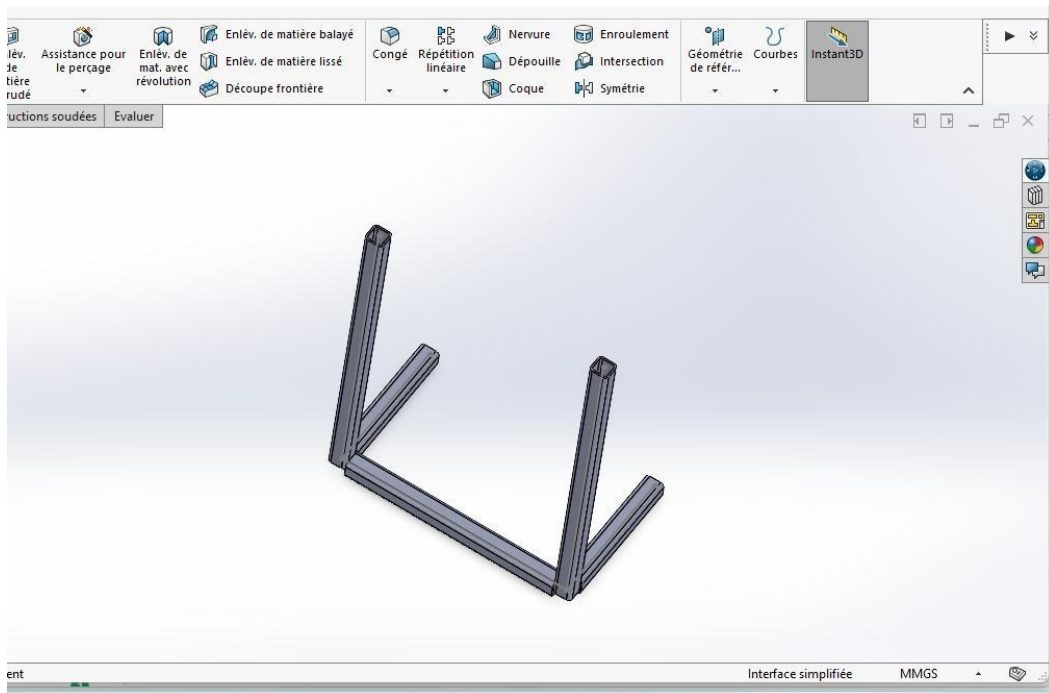


Figure 3.2 - Châssis porteur modélisé sous SolidWorks

- les montants verticaux assurent la tenue du système et servent de référence de montage ;
- les traverses inférieures stabilisent l'ensemble et limitent le basculement ;

- les liaisons mécaniques doivent être serrées sans déformer les profilés ;
- l'équerrage du châssis doit être vérifié avant le montage des éléments mobiles.

3.4 Guidage et éléments linéaires

Le guidage est un point critique du robot. Un mauvais alignement provoque des frottements, des points durs et une perte de précision. La conception combine des profilés de guidage, des tiges cylindriques, des supports et des roulements linéaires afin d'obtenir un mouvement régulier avec des composants simples à intégrer.

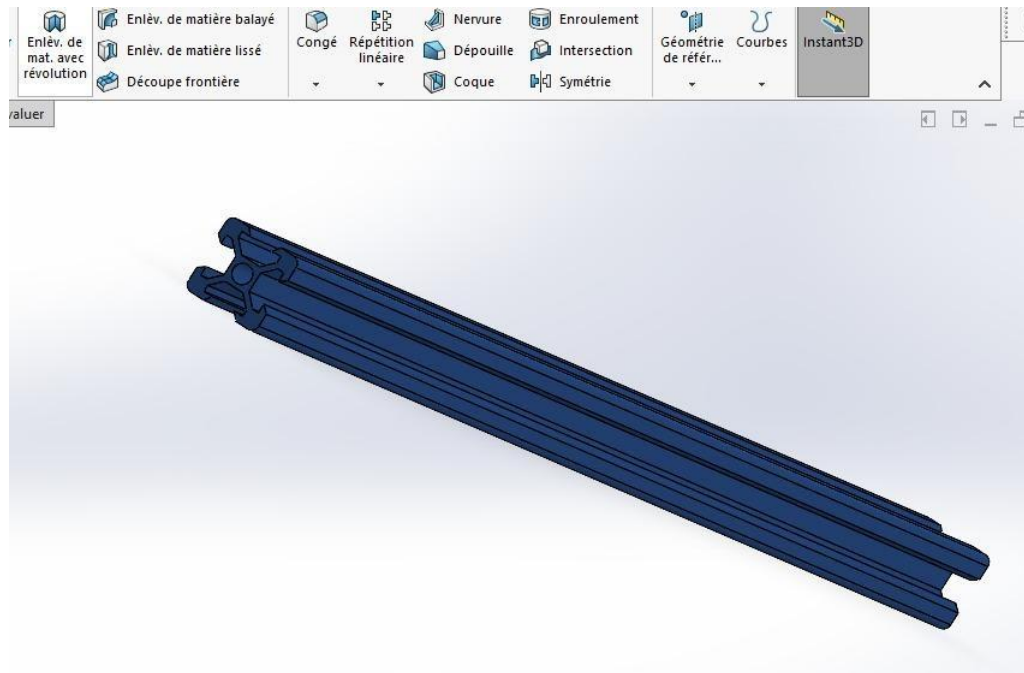


Figure 3.3 - Profilé V-Slot utilisé pour le guidage du chariot

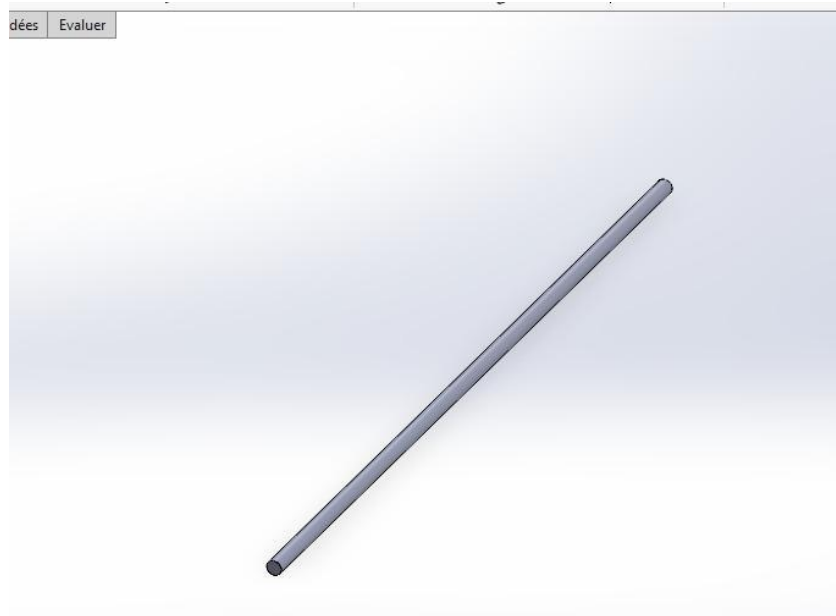


Figure 3.4 - Tige cylindrique utilisée comme élément de guidage

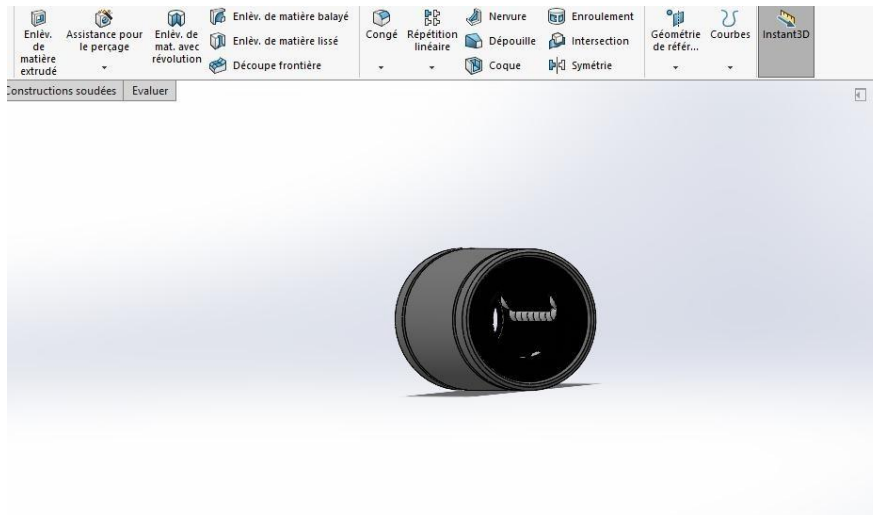


Figure 3.5 - Roulement linéaire utilisé pour réduire les frottements de translation

Les tiges cylindriques doivent être parallèles et solidement fixées. Les roulements linéaires assurent le glissement entre la partie mobile et les tiges, tandis que le profilé V-Slot permet de guider le chariot par contact avec les roues. Le réglage de ces éléments conditionne directement la précision du mouvement.

3.5 Supports et pièces de fixation

Les supports ont pour rôle de maintenir les tiges, les roulements et les sous-ensembles mobiles dans une position reproductible. Leur géométrie doit assurer le passage des vis de fixation, la retenue des éléments cylindriques et l'accès aux zones de réglage. Les pièces ci-dessous ont été intégrées dans la CAO afin de valider leur compatibilité avec le châssis.

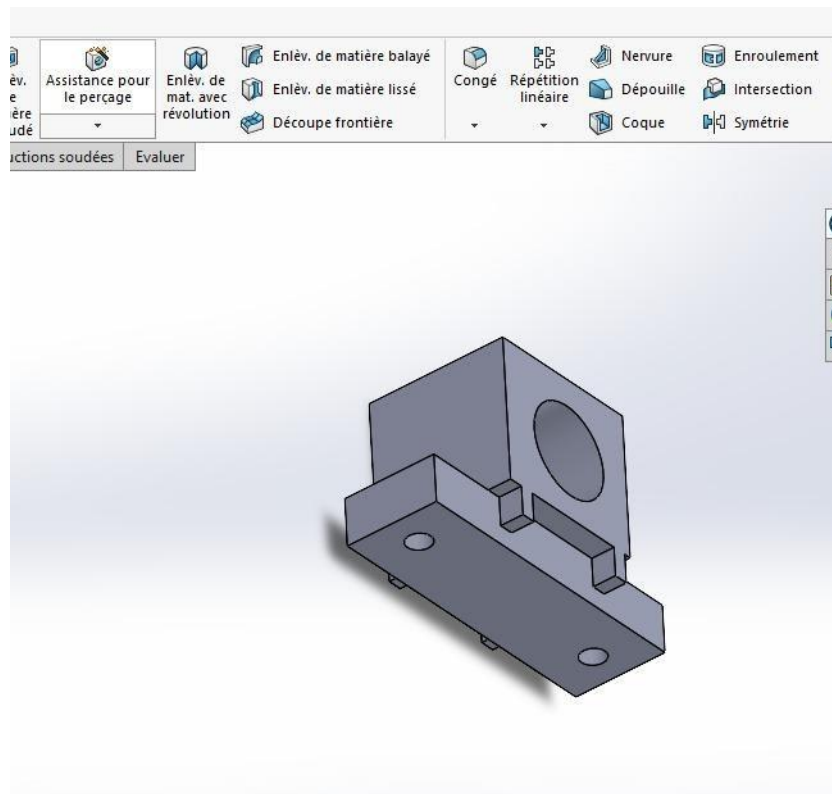


Figure 3.6 - Support mécanique de guidage et de fixation

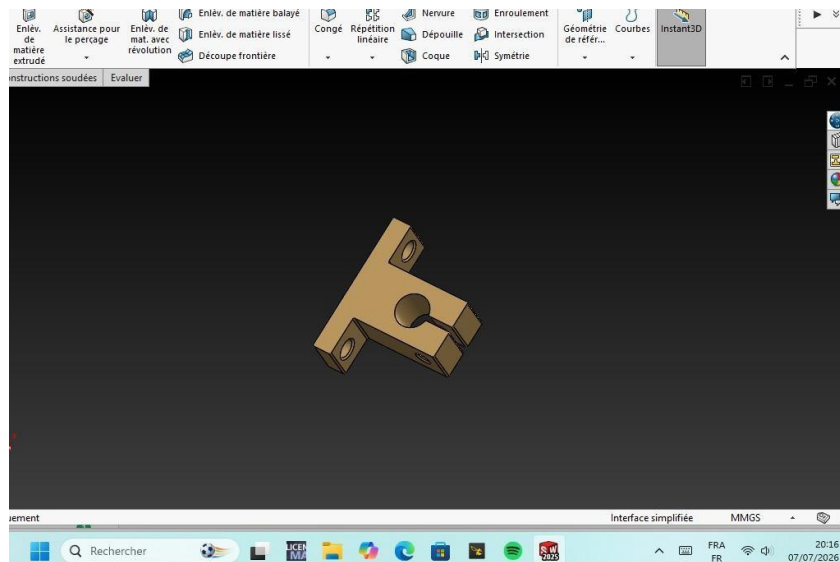


Figure 3.7 - Pièce de fixation utilisée pour l'intégration des éléments de guidage

- le support doit immobiliser la tige sans créer de désalignement ;
- les perçages doivent permettre un serrage fiable et accessible ;
- la géométrie doit éviter les interférences avec le chariot mobile ;
- la pièce doit rester suffisamment rigide pour conserver la précision du guidage.

3.6 Chariot mobile et transmission H-Bot

Le chariot mobile constitue l'élément qui reçoit le mouvement transmis par le système H-Bot. Il doit être léger, rigide et correctement guidé afin de transporter le module de récupération sans vibration excessive. Le principe H-Bot impose une attention particulière à la tension de la courroie et à la symétrie des efforts, car un défaut de réglage peut entraîner une inclinaison du chariot.

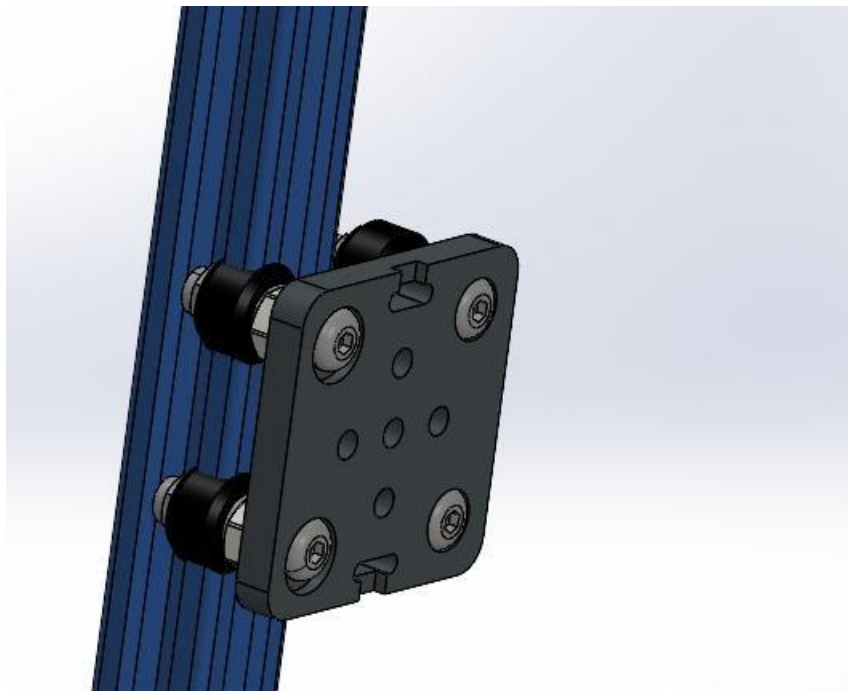


Figure 3.8 - Détail du chariot monté sur le profilé de guidage

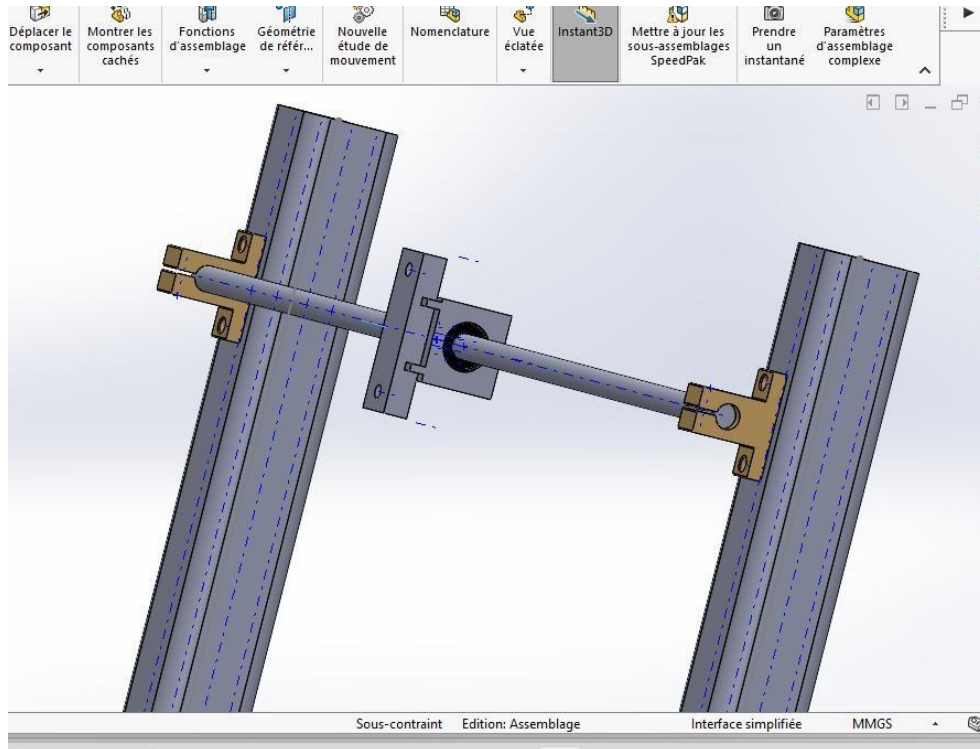


Figure 3.9 - Liaison entre tige de guidage, support et chariot mobile

La transmission doit permettre un déplacement fluide sans glissement. Les galets, la platine et les points de fixation doivent être alignés pour réduire les efforts parasites. Pendant les essais, les réglages principaux concernent la tension de courroie, le parallélisme des rails et le serrage des roues de guidage.

3.7 Assemblage CAO du système

L'assemblage SolidWorks permet de vérifier l'intégration des composants avant fabrication ou montage réel. Les vues d'ensemble servent à contrôler l'encombrement, la position des tiges, le placement des supports et la mobilité du chariot. Elles remplacent les anciens schémas génériques par des représentations directement liées au prototype.

Chapitre 3 : Conception mécanique et électronique du robot ASRS

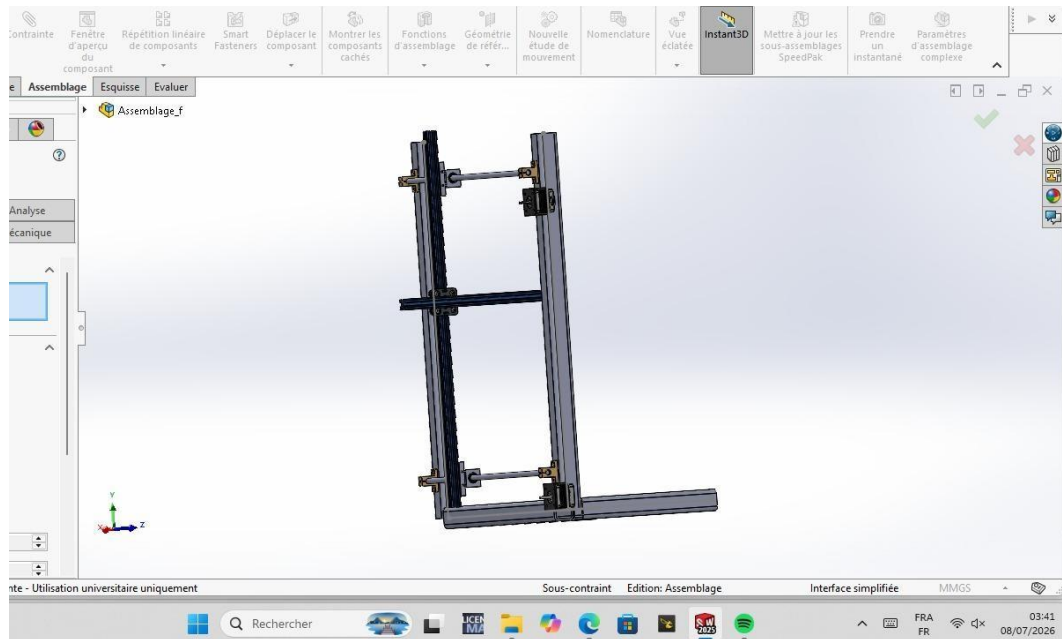


Figure 3.10 - Vue d'ensemble de l'assemblage mécanique du robot



Figure 3.11 - Vue latérale de l'assemblage avec guidage et supports

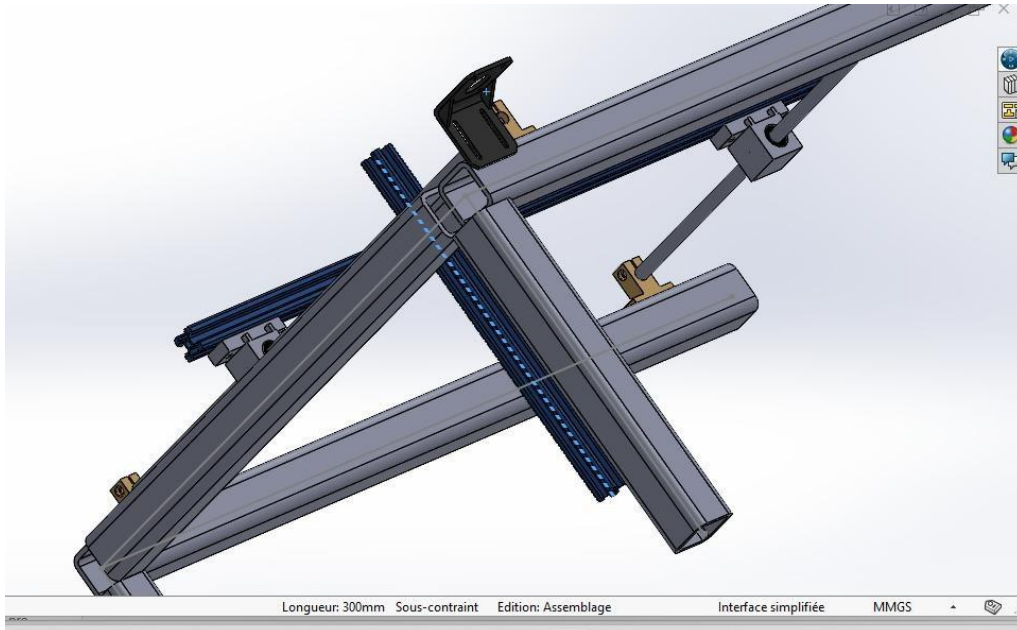


Figure 3.12 - Vue de dessous montrant l'intégration des profilés et du guidage

Ces vues confirment la cohérence de l'architecture mécanique : la base assure la stabilité, les montants portent le guidage vertical, et le chariot reste accessible pour les opérations de réglage. La conception conserve une logique de montage simple, ce qui est important pour un prototype destiné aux tests.

3.8 Points de validation mécanique

La validation mécanique ne se limite pas à l'aspect visuel du modèle 3D. Elle consiste à vérifier que les mouvements sont possibles, que les composants ne se heurtent pas et que les réglages restent accessibles après assemblage. Les critères suivants doivent être contrôlés avant les essais automatisés :

- absence d'interférences entre le chariot, les supports et le châssis ;
- déplacement fluide du chariot sur toute la course utile ;
- alignement des tiges et des profilés de guidage ;
- tension correcte de la courroie du système H-Bot ;
- serrage fiable des pièces sans blocage du mouvement ;
- accessibilité des vis de réglage et de maintenance.

3.9 Synthèse de la conception mécanique

La conception mécanique retenue repose sur une architecture H-Bot simple, modulaire et adaptée à la réalisation d'un prototype ASRS. La nouvelle présentation du chapitre met en avant les éléments réellement modélisés sous SolidWorks et supprime les schémas génériques qui ne représentaient pas fidèlement le robot. Les photos intégrées montrent les pièces principales, les guidages et l'assemblage global, ce qui rend la partie mécanique plus claire et directement liée au système réalisé.

3.10 Architecture électronique et commande

Introduction de la partie électronique

Après la conception mécanique de la plateforme Sky Pharma, ce chapitre présente la conception électronique du système. La partie électronique assure le lien entre la commande logicielle et les éléments physiques du robot. Elle permet de piloter les moteurs pas à pas, de gérer les fins de course, d'assurer l'alimentation des composants et de garantir un fonctionnement fiable et sécurisé.

Le système électronique retenu repose principalement sur une carte **Arduino UNO**, une **CNC Shield V3**, des drivers **DRV8825**, des moteurs pas à pas NEMA 17, des capteurs de **fin de course**, une alimentation principale de 24 V et un système de refroidissement destiné aux drivers.

L'objectif de ce chapitre est de présenter le rôle de chaque composant, de justifier les choix effectués, de décrire le câblage général et d'identifier les précautions nécessaires pour éviter les erreurs de fonctionnement.

3.10.1 Architecture électronique globale

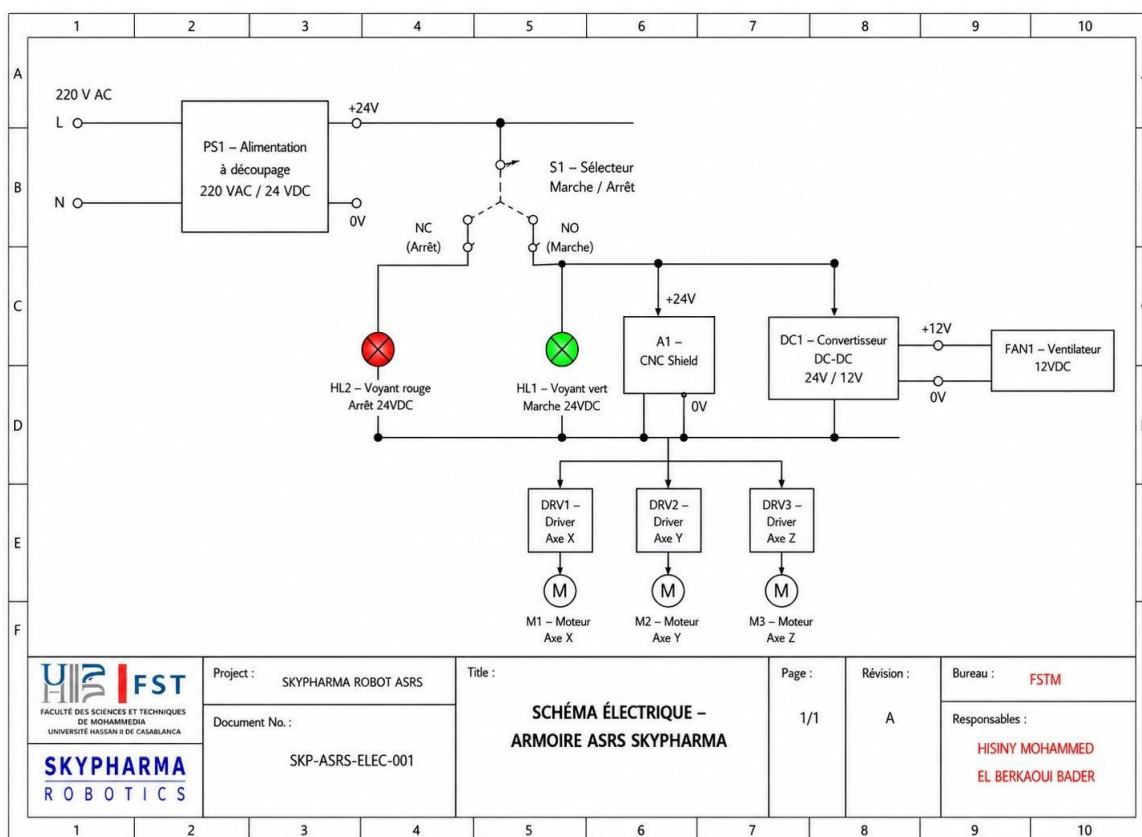


Figure 3.13 - Schéma électrique de l'armoire ASRS SkyPharma

L'architecture électronique de Sky Pharma est organisée autour d'une chaîne de commande simple :

Architecture électronique globale



Figure 3.14 - Architecture électronique globale de SkyPharma

le système est piloté depuis un ordinateur ou un serveur, qui génère les trajectoires sous forme de **G-code**. Ces instructions sont transmises à l'**Arduino UNO** via une liaison USB. L'Arduino, équipé du firmware **GRBL**, joue un rôle central : il interprète les commandes reçues et les convertit en signaux de commande adaptés au pilotage des axes.

La **CNC Shield V3** est montée directement sur l'Arduino et sert d'interface matérielle entre la logique de commande et les éléments de puissance. Elle facilite la connexion des drivers moteurs, des fins de course ainsi que de l'alimentation externe. Elle assure également la distribution des signaux de commande vers les différents axes.

Les drivers **DRV8825** reçoivent les signaux STEP (impulsions de pas) et DIR (direction) générés par l'Arduino. Ils amplifient ces signaux pour alimenter correctement les bobines des moteurs pas à pas, permettant ainsi un contrôle précis de la position et de la vitesse.

Enfin, les moteurs pas à pas convertissent ces signaux électriques en mouvements mécaniques, assurant le déplacement des différents axes du système.

3.10.2 Carte Arduino UNO

Rôle de l'Arduino UNO

L'**Arduino UNO** est la carte de commande principale du système. Elle reçoit les instructions depuis le PC via le port USB, puis génère les signaux de commande nécessaires au pilotage des moteurs.

Dans ce projet, l'**Arduino UNO** n'est pas programmé manuellement pour générer directement chaque impulsion moteur. Il utilise le firmware **GRBL**, qui transforme les commandes **G-code** en signaux électriques adaptés aux drivers moteurs.

Justification du choix

Le choix de l'**Arduino UNO** est justifié par plusieurs raisons :

- disponibilité ;
- coût réduit ;
- compatibilité avec la CNC Shield V3 ;
- compatibilité avec GRBL ;
- facilité de programmation ;
- documentation abondante ;

- intégration simple avec Python via le port série.
- L'Arduino UNO constitue donc une solution adaptée pour un prototype académique de machine à commande numérique légère.

Rôle dans la chaîne de commande

L'Arduino assure les fonctions suivantes :

1. réception du G-code ;
2. interprétation des déplacements ;
3. conversion des consignes en signaux STEP/DIR ;
4. génération des impulsions STEP ;
5. génération des signaux DIR ;
6. gestion des fins de course ;
7. exécution du homing ;
8. arrêt en cas de limite ou d'erreur ;
9. communication avec le logiciel.

3.10.3 CNC Shield V3

Présentation

La **CNC Shield V3** est une carte d'extension qui se monte directement sur l'**Arduino UNO**. Elle permet de connecter facilement les drivers moteurs, les moteurs pas à pas, les fins de course et l'alimentation.

Elle est largement utilisée dans les petites machines CNC, les graveuses laser et les imprimantes 3D simples.

Rôle dans Sky Pharma

Dans Sky Pharma, la **CNC Shield V3** assure les fonctions suivantes :

- support des drivers DRV8825 ;
- distribution des signaux STEP et DIR ;
- connexion des moteurs X, Y et Z ;
- connexion des fins de course ;
- distribution de l'alimentation moteur ;
- simplification du câblage.
- Elle évite de réaliser une carte électronique spécifique et réduit fortement les erreurs de câblage.

Axes utilisés

Le système utilise trois axes :

Axe	Fonction mécanique	Moteur associé
X	Déplacement horizontal	NEMA 17 / 17HS4401
Y	Déplacement transversal	NEMA 17 / 17HS4401

Axe	Fonction mécanique	Moteur associé
Z	Mouvement du plateau	NEMA 17 / 17HS4023

Tableau 3.2 - Fonctions mécaniques du prototype

Dans une architecture **H-Bot**, les axes X et Y sont pilotés par deux moteurs dont les mouvements sont combinés. Le paramétrage logiciel devra tenir compte du sens réel de rotation des moteurs.

3.10.4 Drivers DRV8825

Rôle du driver

Le **DRV8825** est un driver de **moteur pas à pas**. Son rôle est de commander le courant dans les bobines du moteur à partir des signaux logiques fournis par l'Arduino.

L'Arduino ne peut pas alimenter directement un **moteur pas à pas**, car ses sorties ne fournissent qu'un faible courant. Le driver **DRV8825** agit donc comme une interface de puissance entre la commande logique et le moteur.

Signaux de commande

Chaque driver reçoit principalement deux signaux :

- STEP : chaque impulsion correspond à un pas ou micro-pas du moteur ;
- DIR : définit le sens de rotation du moteur.

Le driver reçoit également l'alimentation moteur via la CNC Shield.

Micro-pas

Le **DRV8825** permet de diviser un pas moteur en micro-pas. Cela améliore la fluidité du mouvement et augmente la résolution du déplacement.

Les modes possibles sont généralement :

Mode	Description
Full step	Pas complet
1/2 step	Demi-pas
1/4 step	Quart de pas
1/8 step	Huitième de pas
1/16 step	Seizième de pas
1/32 step	Trente-deuxième de pas

Tableau 3.3 - Modes de fonctionnement ou de commande

Dans Sky Pharma, le choix du micro-pas doit être un compromis entre précision, couple disponible et vitesse. Un micro-pas élevé améliore la résolution, mais peut réduire le couple utile.

Réglage du courant

Le réglage du courant est une étape essentielle. Si le courant est trop faible, le moteur peut perdre des pas. Si le courant est trop élevé, le moteur et le driver peuvent chauffer.

Le courant se règle à l'aide du potentiomètre présent sur le driver. Le réglage doit être réalisé avec précaution, avant les tests dynamiques.

3.11 Motorisation, capteurs et alimentation

Moteurs pas à pas utilisés

Les moteurs utilisés dans le projet sont des moteurs pas à pas NEMA 17 :

- moteurs X et Y : 17HS4401 ;
- moteur Z : 17HS4023.

Ces moteurs sont adaptés aux systèmes de positionnement car ils permettent un contrôle précis de la position à partir d'impulsions électriques.

Connexion des moteurs

Chaque **moteur pas à pas** possède deux bobines. Il est important d'identifier correctement les paires de fils correspondant à chaque bobine avant le branchement sur la CNC Shield.

Une mauvaise connexion peut provoquer :

- vibration du moteur sans rotation ;
- échauffement anormal ;
- rotation irrégulière ;
- perte de pas ;
- impossibilité de déplacement.

Pour identifier les bobines, on peut utiliser un multimètre en mode continuité ou résistance.

Sens de rotation

Le sens de rotation d'un moteur peut être inversé de deux manières :

- inversion d'une paire de fils du moteur ;
- modification du paramétrage logiciel dans GRBL.

Dans un premier montage, il est préférable de corriger le sens dans le logiciel lorsque cela est possible, afin d'éviter de multiplier les modifications de câblage.

3.11.1 Fins de course

Rôle des fins de course

Les fins de course sont des capteurs utilisés pour détecter les positions limites ou les positions de référence du robot. Ils sont indispensables pour réaliser le **homing** et définir l'origine machine.

Dans Sky Pharma, les fins de course permettent :

- de référencer les axes ;
- de protéger le robot contre les déplacements hors zone ;
- d'arrêter le mouvement en cas de dépassement ;
- d'améliorer la sécurité du système ;
- de garantir une position initiale connue.

Positionnement

Les fins de course doivent être placées aux extrémités des axes ou aux positions de référence choisies. Le positionnement doit être mécanique et logique.

Exemple :

Axe	Position du fin de course	Fonction
X	Extrémité gauche ou droite	Référence X
Y	Extrémité avant ou arrière	Référence Y
Z	Position haute ou basse	Référence Z

Tableau 3.4 - Affectation des axes et composants

Le choix de la position dépend de l'architecture réelle du robot et du sens de **homing** choisi dans **GRBL**.

Type de câblage

Les fins de course peuvent être câblées en mode normalement ouvert ou normalement fermé. Le mode normalement fermé est souvent préféré pour la sécurité, car une coupure de fil peut être détectée comme un défaut.

Dans un prototype, il est possible de commencer avec un câblage simple, mais il faut vérifier que le signal est bien reconnu par **GRBL** avant d'activer les limites matérielles.

3.11.2 Alimentation électrique

Alimentation principale 24 V

Le système est alimenté par une alimentation principale de 24 V. Cette tension est utilisée pour alimenter la partie puissance des moteurs à travers la CNC Shield et les drivers **DRV8825**.

Le choix du 24 V présente plusieurs avantages :

- meilleure réponse dynamique des moteurs ;
- réduction du courant pour une même puissance ;
- meilleure stabilité à vitesse élevée ;
- compatibilité avec de nombreux systèmes CNC légers.
- Il faut cependant respecter les limites des composants utilisés et vérifier que les drivers acceptent la tension choisie.

Distribution de l'alimentation

L'alimentation 24 V est distribuée vers la CNC Shield pour alimenter les moteurs. L'Arduino peut être alimenté par USB depuis le PC pendant les essais, ou par une alimentation adaptée selon la configuration finale.

- Il est important de séparer clairement :
- la partie puissance : moteurs, drivers, alimentation 24 V ;
- la partie commande : Arduino, signaux logiques, USB ;
- les accessoires : ventilateur, éventuels capteurs ou modules.

Convertisseur Buck

Un convertisseur Buck peut être utilisé pour abaisser la tension de 24 V vers une tension plus faible, par exemple 12 V ou 5 V, selon les besoins.

Dans Sky Pharma, un Buck peut servir à alimenter :

- un ventilateur 12 V ;
- certains capteurs ;
- des modules auxiliaires ;
- éventuellement une carte de commande secondaire.
- Le convertisseur doit être dimensionné selon le courant demandé par les charges connectées.

3.11.3 Gestion thermique

Problème d'échauffement

Les drivers **DRV8825** peuvent chauffer pendant le fonctionnement, surtout lorsque le courant moteur est élevé ou lorsque les moteurs restent alimentés à l'arrêt.

Un échauffement excessif peut provoquer :

- perte de couple ;
- arrêt temporaire du driver ;
- fonctionnement instable ;
- diminution de la durée de vie du composant.

Dissipateurs thermiques

Des dissipateurs thermiques sont ajoutés sur les drivers afin d'améliorer l'évacuation de la chaleur. Ils doivent être bien collés sur la puce principale du driver.

Ventilation forcée

Un ventilateur 12 V DC est ajouté pour refroidir la zone de la CNC Shield et des drivers. Ce ventilateur peut être alimenté à partir du convertisseur Buck 24 V vers 12 V.

La ventilation permet de maintenir une température plus stable pendant les cycles répétés.

Validation thermique

La validation thermique peut être réalisée en observant la température des drivers pendant un cycle de test :

- test à vide ;
- test avec charge ;
- test de plusieurs cycles successifs ;
- observation de la stabilité du mouvement ;
- vérification de l'absence d'arrêt des drivers.

3.12 Schéma électrique et câblage de l'armoire

Organisation du câblage

Le câblage doit être organisé de manière claire afin de faciliter la maintenance et d'éviter les erreurs.

Les règles principales sont :

- séparer les câbles de puissance et les câbles de signal ;
- éviter les fils trop longs ;
- fixer les câbles pour éviter leur contact avec les parties mobiles ;
- utiliser des borniers fiables ;
- repérer les fils par couleur ou étiquette ;
- vérifier la polarité avant la mise sous tension.

Connexions principales

Élément	Connexion
Arduino UNO	USB vers PC
CNC Shield V3	Enfichée sur Arduino
DRV8825	Slots X, Y, Z de la CNC Shield
Moteur X	Sortie moteur X
Moteur Y	Sortie moteur Y
Moteur Z	Sortie moteur Z
Fins de course	Entrées Endstop
Alimentation 24 V	Entrée puissance CNC Shield
Ventilateur 12 V	Sortie Buck 12 V
LED verte	Signalisation de fonctionnement
LED noire	Signalisation d'arrêt ou défaut

Tableau 3.5 - Connexions principales de l'armoire

3.13 Validation électromécanique

Test de communication

Le premier test consiste à vérifier la communication entre le PC et l'Arduino. Si **GRBL** est correctement installé, la carte doit répondre via le moniteur série ou un logiciel de contrôle.

Test des moteurs

Chaque axe est testé séparément :

- déplacement de l'axe X ;
- déplacement de l'axe Y ;
- déplacement de l'axe Z ;
- vérification du sens de rotation ;
- vérification de la fluidité du mouvement ;
- vérification de l'absence de vibration excessive.

Test des fins de course

Les fins de course sont testées avant l'activation du **homing**. Il faut vérifier que chaque capteur est reconnu correctement par **GRBL**.

Test de fonctionnement complet

Après validation individuelle des composants, un test complet est réalisé :

1. mise sous tension ;
2. homing ;
3. déplacement vers une position donnée ;
4. activation du cycle de récupération ;
5. retour à la position initiale ;
6. arrêt du système.

Chapitre 4 : Développement embarqué et pilotage du robot

4.1 Introduction

Après la conception mécanique et électronique de la plateforme Sky Pharma, ce chapitre présente la partie embarquée du système. Cette partie constitue le lien direct entre le logiciel de commande et le robot physique. Elle permet de transformer les ordres générés par l'application en mouvements réels des moteurs.

Le développement embarqué repose principalement sur l'utilisation du firmware **GRBL**, installé sur une carte **Arduino UNO**. **GRBL** permet d'interpréter des commandes **G-code** et de générer les signaux nécessaires au pilotage des moteurs pas à pas via la **CNC Shield V3** et les drivers **DRV8825**.

Dans le cadre du projet Sky Pharma, le rôle de la partie embarquée est d'assurer :

- la réception des commandes envoyées par le logiciel ;
- l'interprétation du G-code ;
- la commande des axes X, Y et Z ;
- la gestion du homing ;
- la gestion des fins de course ;
- la limitation des déplacements ;
- la sécurité du robot ;
- le retour d'état vers le logiciel.
- Ce chapitre détaille donc la configuration de GRBL, le paramétrage des axes, le calcul des steps/mm, la logique de déplacement, la communication série et les tests de validation.

4.2 Rôle de la partie embarquée dans Sky Pharma

La partie embarquée constitue le cœur de commande du robot Sky Pharma. Elle joue le rôle d'intermédiaire entre le logiciel (qui décide des actions) et les composants électroniques de puissance (qui exécutent les mouvements).

Concrètement, elle reçoit des instructions depuis le PC sous forme de commandes numériques, puis les transforme en signaux électriques précis permettant de piloter les moteurs pas à pas.

Principe de fonctionnement

Le fonctionnement global du système peut être représenté par la chaîne suivante :

Chaîne de pilotage embarquée du robot

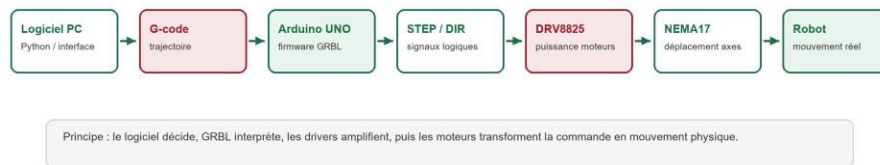


Figure 4.1 - Chaîne de pilotage embarquée du logiciel vers le mouvement robot

1. Explication du schéma :

1. Le logiciel (sur PC) génère des commandes de déplacement (ex : aller à une position).
2. Ces commandes sont traduites en G-code, un langage standard utilisé en robotique et CNC.
3. L'Arduino équipé du firmware GRBL reçoit ces commandes via USB.
4. GRBL interprète le G-code et génère des signaux électriques :
STEP : impulsions pour faire avancer le moteur
DIR : direction du mouvement
5. Les drivers DRV8825 amplifient ces signaux pour alimenter correctement les moteurs.
6. Les moteurs pas à pas exécutent le mouvement demandé.

2. Rôle essentiel de la partie embarquée :

La partie embarquée assure plusieurs fonctions importantes :

- conversion des commandes logicielles en actions physiques ;
- synchronisation des mouvements des axes ;
- gestion des vitesses et accélérations ;
- contrôle précis des positions ;
- communication avec le logiciel ;
- gestion des sécurités (fins de course, limites).

3. Avantages de cette architecture :

Cette organisation présente plusieurs avantages :

- simplification du développement logiciel (le logiciel envoie seulement des ordres) ;
- meilleure précision et fiabilité des mouvements ;
- gestion automatique des accélérations et décélérations ;
- utilisation d'un standard industriel (G-code) ;
- facilité de maintenance et de diagnostic ;
- modularité du système (chaque partie peut être améliorée séparément).

Ainsi, la partie embarquée permet de transformer une simple commande informatique en un mouvement réel précis du robot Sky Pharma.

Figure 4.2 - Architecture du développement embarqué

Architecture du pilotage embarqué



4.3 Choix du firmware GRBL

4.3.1 Présentation de GRBL

GRBL est un firmware open-source conçu pour transformer une carte **Arduino UNO** en contrôleur de machine CNC. Il interprète des commandes **G-code** et génère des signaux STEP et DIR pour piloter des moteurs pas à pas.

Il est généralement utilisé dans :

- les petites machines CNC ;
- les graveuses laser ;
- les traceurs XY ;
- les machines de découpe ;
- les robots cartésiens simples ;
- les prototypes mécatroniques.

Dans Sky Pharma, **GRBL** est utilisé non pas pour usiner une pièce, mais pour piloter le déplacement du robot vers les emplacements des **médicaments**.

4.3.2 Justification du choix

Le choix de **GRBL** est justifié par plusieurs raisons :

- compatibilité avec Arduino UNO ;
- compatibilité avec CNC Shield V3 ;
- commande standard par G-code ;
- gestion intégrée des moteurs pas à pas ;
- gestion du homing ;
- gestion des limites matérielles et logicielles ;
- réglage des vitesses et accélérations ;
- communication simple via USB série ;

- fiabilité suffisante pour un prototype académique.
- L'utilisation de GRBL permet d'éviter de développer entièrement un programme bas niveau pour générer les impulsions moteurs. Le travail consiste plutôt à configurer correctement les paramètres du système et à générer les bons ordres de déplacement.

4.3.3 Limites de GRBL

GRBL est très pratique, mais il possède certaines limites :

- il est prévu principalement pour des machines cartésiennes ;
- il ne gère pas directement toutes les architectures spéciales sans adaptation ;
- il nécessite un bon paramétrage des axes ;
- il ne détecte pas automatiquement une perte de pas moteur ;
- il dépend de la qualité mécanique du système ;
- il ne remplace pas les capteurs de sécurité.

4.4 Installation et configuration initiale de GRBL

4.4.1 Installation du firmware

L'installation de **GRBL** sur **Arduino UNO** consiste à téléverser le firmware à l'aide de l'environnement Arduino IDE ou d'un outil équivalent. Une fois le programme installé, l'Arduino devient un contrôleur **GRBL** capable de recevoir des commandes série.

Les étapes générales sont :

1. installer Arduino IDE ;
2. ajouter la bibliothèque GRBL ;
3. ouvrir l'exemple grblUpload ;
4. sélectionner la carte Arduino UNO ;
5. sélectionner le port COM ;
6. téléverser le firmware ;
7. ouvrir le moniteur série ;
8. régler la vitesse de communication ;
9. vérifier la réponse de GRBL.

Après le téléversement, **GRBL** doit répondre avec un message d'initialisation indiquant qu'il est prêt à recevoir des commandes.

4.4.2 Communication série

La communication entre le PC et l'Arduino se fait via USB. Du point de vue logiciel, cette liaison est considérée comme une liaison série.

Le logiciel envoie une ligne de commande **G-code**, puis attend la réponse de **GRBL**. Les réponses les plus courantes sont :

Réponse GRBL	Signification
ok	Commande acceptée et exécutée
error	Commande incorrecte ou non valide
ALARM	État d'alarme, souvent lié au homing ou aux limites
Idle	Machine au repos
Run	Machine en mouvement
Hold	Mouvement suspendu
Door	Sécurité ou interruption selon configuration

Tableau 4.1 - Réponses GRBL et interprétation

La réponse ok est très importante, car elle confirme que **GRBL** a accepté la commande envoyée.

4.5 Organisation des axes du robot

4.5.1 Axes logiques

Dans le système de commande, on utilise des axes dits logiques, qui correspondent à la manière intuitive de décrire les déplacements dans l'espace :

X : déplacement horizontal de gauche à droite

Y : déplacement horizontal d'avant en arrière

Z : déplacement vertical de haut en bas

Ces axes sont ceux utilisés dans le **G-code** et dans le logiciel de pilotage. Ils permettent de définir facilement une position cible dans l'espace sans se soucier de la mécanique interne du robot.

4.5.2 Axes physiques

Au niveau matériel, le robot est piloté par trois moteurs :

- moteur A
- moteur B
- moteur Z

Contrairement à une architecture classique (type cartésienne), les moteurs A et B ne sont pas directement associés aux axes X et Y. Ils sont reliés à un système de courroies qui combine leurs mouvements pour produire les déplacements réels.

Le moteur Z, en revanche, est directement responsable du mouvement vertical.

4.5.3 Architecture H-Bot

Le robot utilise une architecture de type **H-Bot**, caractérisée par un système de courroies en forme de "H". Dans cette configuration :

- les moteurs A et B travaillent simultanément pour générer les mouvements dans le plan XY ;
- aucun moteur ne correspond directement à un seul axe (X ou Y) ;
- chaque déplacement résulte d'une combinaison des rotations des deux moteurs.

Par exemple :

- si les deux moteurs tournent dans le même sens, le robot se déplace selon un axe ;
- s'ils tournent en sens opposé, le déplacement se fait selon l'autre axe.

Cette architecture permet de réduire le nombre de moteurs tout en conservant une bonne précision, mais elle nécessite une transformation mathématique entre les axes logiques et les moteurs.

4.5.4 Approche adoptée

Approche retenue : modification du code source **GRBL**

Pour simplifier l'utilisation du robot, la transformation entre les axes logiques (X, Y) et les moteurs (A, B) est directement intégrée dans le firmware **GRBL**.

La relation utilisée est la suivante :

- $\Delta A = \Delta X + \Delta Y$
- $\Delta B = \Delta X - \Delta Y$

Cela signifie que :

- pour effectuer un déplacement selon X ou Y, GRBL calcule automatiquement les mouvements nécessaires des moteurs A et B ;
- le logiciel de contrôle continue d'envoyer des commandes classiques en coordonnées cartésiennes (X, Y, Z) ;
- toute la complexité liée à l'architecture H-Bot est gérée au niveau du firmware.

Cette approche présente plusieurs avantages :

- elle simplifie le développement logiciel (pas besoin de gérer la cinématique côté application).
- elle rend le système plus robuste et plus facile à maintenir .
- elle permet d'utiliser des outils standards compatibles G-code sans modification.

4.6 Système de coordonnées

- Le système de coordonnées constitue un élément fondamental du pilotage du robot, car il permet de relier les positions physiques réelles aux instructions numériques envoyées au contrôleur GRBL.
- coordonnées machine après homing : Après la procédure de homing, le robot se positionne sur une référence physique connue, généralement définie par les fins de course. À partir de ce point, GRBL initialise les coordonnées machine (Machine Coordinates), souvent notées (X=0, Y=0). Ces coordonnées servent de base absolue

pour tous les déplacements ultérieurs et garantissent une répétabilité précise des mouvements.

- coordonnées des emplacements stockées en base : Chaque emplacement de stockage (cases de médicaments, zones de dépôt, etc.) est associé à des coordonnées spécifiques enregistrées dans une base de données. Ces coordonnées sont définies lors de la phase de calibration et correspondent à des positions physiques précises dans l'espace de travail du robot. Cela permet au système logiciel de sélectionner dynamiquement une position cible en fonction de la demande utilisateur.
- utilisation directe des coordonnées réelles : Le système utilise directement les coordonnées réelles pour générer les commandes G-code, sans transformation complexe intermédiaire. Cela simplifie l'architecture logicielle et réduit les risques d'erreurs de conversion. Lorsqu'un médicament est demandé, le logiciel récupère les coordonnées correspondantes et envoie une instruction de déplacement (par exemple G0 ou G1) vers cette position.
- distinction entre coordonnées machine et coordonnées de travail : Bien que les coordonnées machine soient utilisées comme référence absolue, il est possible de définir des coordonnées de travail (Work Coordinates) pour faciliter certaines opérations. Ces coordonnées permettent de décaler l'origine selon un repère local, ce qui peut être utile pour des tâches spécifiques ou des ajustements temporaires.
- gestion des offsets et calibration : Des offsets peuvent être appliqués pour corriger les écarts mécaniques ou les imprécisions de montage. Ces ajustements sont déterminés lors de la calibration initiale et peuvent être mis à jour si nécessaire afin de maintenir la précision du système.
- importance pour la précision et la sécurité : Un système de coordonnées bien défini garantit que le robot atteint exactement les positions souhaitées sans collision. Il contribue également à la sécurité globale en évitant les déplacements imprévus ou hors zone autorisée.

4.7 Calcul des steps/mm et intégration avec la plateforme Python

4.7.1 Définition

Le nombre de steps/mm représente le nombre d'impulsions envoyées au moteur pour parcourir 1 mm. Ce paramètre est essentiel pour assurer la précision des déplacements du robot.

Dans notre projet, l'Arduino exécutant **GRBL** ne comprend que des commandes **G-code**. Nous avons donc développé une plateforme en **Python**, similaire à UGS (Universal **G-code** Sender), qui permet de générer et envoyer ces commandes tout en contrôlant précisément les déplacements de chaque moteur.

4.7.2 Exemple courroie GT2

$$\text{Steps/mm} = (200 \times 16) / (20 \times 2) = 80$$

Ce calcul est utilisé dans notre plateforme **Python** pour convertir les distances demandées en **G-code** (ex : G1 X10) en mouvements réels exécutés par les moteurs.

4.7.3 Axe Z

$\text{Steps/mm} = (200 \times 16) / 8 = 400$ L'axe Z utilise généralement une vis, ce qui nécessite un calcul différent. Ces valeurs sont configurées dans **GRBL** et utilisées automatiquement lors de l'exécution des commandes envoyées depuis notre interface **Python**.

4.7.4 Correction

$$\text{Steps_corrigé} = \text{Steps_actuel} \times (\text{Distance_demandée} / \text{Distance_réelle})$$

Grâce à notre plateforme **Python**, nous pouvons tester les déplacements en envoyant des commandes **G-code** et mesurer les erreurs. Cela permet d'ajuster précisément les steps/mm afin d'obtenir une correspondance exacte entre la distance demandée et la distance réelle parcourue.

4.8 Paramètres GRBL principaux

Commande :

\$\$

Commande	Rôle
\$\$	Afficher les paramètres GRBL
\$H	Lancer le homing
\$X	Déverrouiller après alarme
?	Lire l'état machine
G90	Passage en coordonnées absolues
G0 / G1	Déplacement rapide ou contrôlé

Tableau 4.2 - Principales commandes GRBL utilisées

Paramètre	Signification
\$100 - \$102	Nombre de pas par millimètre (steps/mm) pour X, Y, Z
\$110 - \$112	Vitesse maximale (mm/min) pour X, Y, Z
\$120 - \$122	Accélération (mm/s²) pour X, Y, Z
\$130 - \$132	Dimensions maximales de déplacement (mm) pour X, Y, Z

Tableau 4.3 - Paramètres GRBL de référence

4.9 Homing et référencement

Commande :

\$H

Rôle :

- définir l'origine machine (position de référence absolue) ;

- sécuriser les mouvements en évitant les collisions ;
- permettre au contrôleur de connaître précisément la position du robot avant toute opération.

Principe de fonctionnement :

Lors de l'exécution de la commande \$H, **GRBL** lance une séquence automatique de référencement. Chaque axe (X et Y dans le cas du robot **H-Bot**) se déplace lentement vers son capteur de **fin de course** (endstop). Dès que le capteur est activé, le mouvement s'arrête, puis un léger recul est effectué pour améliorer la précision (phase de "pull-off").

Une fois tous les axes référencés, la position atteinte est définie comme l'origine machine (coordonnées 0,0). À partir de ce point, tous les déplacements sont calculés de manière fiable.

Importance dans le système Sky Pharma :

- garantit que le robot démarre toujours depuis une position connue ;
- évite les erreurs de positionnement lors de la récupération des médicaments ;
- indispensable après chaque mise sous tension ou perte de position ;
- améliore la répétabilité et la précision globale du système.

Sans **homing**, le robot fonctionnerait en position relative uniquement, ce qui pourrait entraîner des décalages cumulés et des erreurs critiques dans un environnement automatisé.

4.10 Génération automatique du G-code

Le logiciel génère automatiquement les commandes à partir de la **base de données** en traduisant les informations logiques (emplacement du **médicament**, coordonnées de **stockage**, type d'action à effectuer) en instructions compréhensibles par le contrôleur **GRBL**. Concrètement, lorsqu'un **médicament** est sélectionné, le système récupère ses coordonnées (X, Y, Z) dans la **base de données**, puis construit une séquence de commandes **G-code** permettant au robot de se déplacer précisément jusqu'à cet emplacement.

Cette génération inclut plusieurs étapes : d'abord le positionnement initial (souvent après un **homing**), ensuite le déplacement rapide vers la zone cible (commande G0), suivi d'un déplacement plus précis (G1) pour atteindre exactement la position souhaitée. Des commandes supplémentaires peuvent être ajoutées pour activer ou désactiver des actionneurs (comme une pince ou un système de prise).

Le processus est entièrement automatisé, ce qui réduit les erreurs humaines et garantit une cohérence dans les mouvements du robot. De plus, le logiciel peut intégrer des vérifications de sécurité, comme la limitation des vitesses ou la validation des coordonnées, afin d'éviter tout risque de collision ou de dépassement des limites mécaniques.

4.11 Communication série avec Python

Principe

La communication entre **Python** et **GRBL** se fait via une liaison série (USB) utilisant un protocole simple basé sur l'envoi de lignes de texte (**G-code**). **Python** agit comme un client qui envoie des commandes, tandis que **GRBL** les interprète et renvoie des réponses.

Le fonctionnement détaillé est le suivant :

Python ouvre un port série (par exemple /dev/ttyUSB0 ou COM3) avec les bons paramètres (baudrate généralement 115200).

Une commande **G-code** (ex : G0 X100 Y50) est envoyée sous forme de chaîne de caractères terminée par un retour à la ligne.

GRBL reçoit la commande, la place dans son buffer interne, puis l'exécute.

Une fois la commande traitée, **GRBL** renvoie la réponse ok pour indiquer que tout s'est bien passé.

En cas d'erreur (commande invalide, dépassement de limites, etc.), GRBL renvoie un message d'erreur explicite, par exemple error:1, error:2 ou un message d'alarme selon la nature du défaut.

Pour garantir un fonctionnement fiable :

- il faut attendre la réponse ok avant d'envoyer la commande suivante afin d'éviter de saturer le buffer ;
- il est possible de lire en continu les messages de statut (?) pour connaître la position et l'état du robot ;
- une gestion des erreurs doit être mise en place côté Python pour réagir en cas de problème (arrêt, reprise, alerte).

Ainsi, la boucle de communication typique est :

- envoyer une commande ;
- lire la réponse ;
- vérifier si c'est ok ou une erreur ;
- continuer ou gérer l'erreur.

4.12 Sécurité embarquée

La sécurité embarquée constitue un élément essentiel du système afin de garantir un fonctionnement fiable, précis et sans risque pour le matériel et l'utilisateur. Elle repose sur plusieurs mécanismes complémentaires intégrés au niveau du firmware GRBL et de la logique de commande.

Les principales mesures prévues sont les suivantes :

- **homing obligatoire** : avant toute opération, le robot effectue une procédure de référencement afin de définir une origine fiable et d'éviter les déplacements incohérents ;

- limites activées : les fins de course empêchent le robot de dépasser ses limites physiques et provoquent l'arrêt du mouvement en cas de dépassement ;

Chapitre 4 : Développement embarqué et pilotage du robot

- vérification des coordonnées : chaque commande de déplacement est contrôlée pour rester dans la zone de travail autorisée ;
- arrêt en cas d'erreur : en présence d'une anomalie, le système interrompt immédiatement les mouvements et passe en état sécurisé.

Ces mécanismes réduisent les risques d'endommagement du robot et les erreurs de manipulation, tout en améliorant la fiabilité d'exploitation du système Sky Pharma.

Chapitre 5 : Développement logiciel, base de données et interfaces de commande

5.1 Introduction

Après avoir présenté la conception mécanique, électronique et embarquée du système Sky Pharma, ce chapitre est consacré à la partie logicielle du projet. Cette partie représente une étape essentielle, car elle assure la liaison entre l'utilisateur, la **base de données** et le robot de distribution. Le système ne se limite donc pas à un robot piloté par des moteurs, mais constitue une plateforme complète dans laquelle une commande utilisateur peut être transformée automatiquement en une action physique de récupération d'un **médicament**.

La partie logicielle développée repose principalement sur le langage **Python**, une **base de données SQLite**, une interface client accessible depuis une tablette ou un navigateur web, ainsi qu'une interface locale utilisée pour la récupération sur site. Le logiciel assure également la génération automatique du **G-code** et la communication avec l'Arduino exécutant **GRBL** à travers une liaison série.

L'objectif de ce chapitre est de présenter l'architecture logicielle globale du système, d'expliquer le rôle de chaque interface, de détailler l'organisation de la **base de données**, puis de montrer comment une commande passée par un client est traitée jusqu'à l'exécution robotisée. Une attention particulière est portée à l'explication du lien entre le choix d'un **médicament**, ses coordonnées enregistrées dans la **base de données** et la trajectoire générée pour le robot **H-Bot**.

5.2 Rôle de la partie logicielle dans Sky Pharma

La partie logicielle constitue le cerveau décisionnel de la plateforme Sky Pharma. Elle permet de relier le monde numérique, représenté par les interfaces et la **base de données**, au monde physique, représenté par le robot, les moteurs, les fins de course et le plateau de récupération. Sans cette couche logicielle, le robot resterait une machine capable de se déplacer, mais incapable de comprendre une commande utilisateur ou de choisir automatiquement l'emplacement d'un **médicament**.

Le rôle principal du logiciel est donc de transformer une demande simple, formulée par un client à travers une interface, en une suite d'opérations techniques. Le client ne voit qu'une interface claire avec des **médicaments**, un panier et un bouton de confirmation. En arrière-plan, le système effectue plusieurs traitements : il lit les informations dans **SQLite**, vérifie la disponibilité du **médicament**, crée une commande, associe le **médicament** à son emplacement physique, génère une séquence **G-code**, envoie cette séquence à **GRBL**, puis met à jour l'état de la commande.

Cette organisation permet d'assurer une continuité complète entre l'expérience utilisateur et l'exécution robotique. Le logiciel joue donc un rôle d'orchestrateur, car il coordonne les différentes couches du système : interface client, **base de données**, logique de commande, communication série et robot physique.

Architecture logicielle de SkyPharma



Figure 5.1 - Architecture logicielle globale de SkyPharma

5.3 Architecture logicielle globale

Figure 5.2 - Chaîne logicielle de commande, base de données et traçabilité

L'architecture logicielle de Sky Pharma est organisée autour de plusieurs modules complémentaires. Le premier module correspond à l'interface client distante. Elle est destinée au client final et permet de consulter les **médicaments** disponibles, de sélectionner un produit, de l'ajouter au panier et de confirmer la commande. Le deuxième module correspond à l'interface locale de récupération. Cette interface est utilisée au niveau de la station afin de superviser la récupération locale, lancer certains cycles et suivre l'état du robot.

La **base de données SQLite** représente le troisième élément important de cette architecture. Elle centralise les informations nécessaires au fonctionnement du système : **médicaments**, quantités disponibles, coordonnées de **stockage**, commandes, lignes de commande et logs robot. Le logiciel **Python** interroge cette **base de données** pour afficher les **médicaments**, enregistrer les commandes et récupérer les positions physiques associées à chaque produit.

Le quatrième module correspond au générateur de **G-code**. Son rôle est de transformer les coordonnées d'un **médicament** en commandes compréhensibles par **GRBL**. Enfin, le module de communication série, basé sur pyserial, assure l'envoi des commandes à l'Arduino et la lecture des réponses retournées par **GRBL**.

L'architecture globale peut donc être décrite comme une chaîne fonctionnelle allant de l'utilisateur vers le robot. Le client interagit avec l'interface, l'interface communique avec la base **SQLite**, le logiciel génère les trajectoires, puis les commandes sont envoyées à Arduino/**GRBL** pour exécuter le mouvement.

Figure 5.3 - Architecture logicielle globale de SkyPharma

5.4 Choix techniques adoptés

Architecture logicielle de SkyPharma



Le choix de **Python** comme langage principal s'explique par sa simplicité, sa flexibilité et sa capacité à intégrer facilement plusieurs fonctions dans un même environnement. **Python** permet de gérer l'interface, la **base de données**, la logique métier, la génération du **G-code** et la communication série avec l'Arduino. Cette polyvalence en fait un choix adapté pour un prototype académique mécatronique, où l'objectif est de réaliser rapidement une solution fonctionnelle et évolutive.

Streamlit a été utilisé pour développer l'interface client web. Ce choix permet de créer rapidement une interface interactive sans avoir besoin de développer une architecture frontend complexe. Grâce à **Streamlit**, l'application peut être lancée sur un PC puis consultée depuis une tablette ou un navigateur connecté au même réseau Wi-Fi. Cette approche est particulièrement adaptée à une démonstration de projet, car elle permet de présenter une interface moderne et accessible sans infrastructure serveur lourde.

SQLite a été retenue comme **base de données** locale. Cette solution est légère, simple à intégrer avec **Python** et suffisante pour un prototype. Contrairement à des systèmes de bases de données plus complexes, **SQLite** ne nécessite pas de serveur séparé. Les données sont **stockées** localement dans un fichier, ce qui facilite le développement, la sauvegarde et le déplacement du projet.

La bibliothèque **pyserial** assure la communication entre **Python** et Arduino. Elle permet d'ouvrir le port série, d'envoyer les commandes **G-code** et de lire les réponses de **GRBL**. Ce choix est logique, car **GRBL** communique naturellement par liaison série USB avec le PC. Grâce à **pyserial**, le logiciel peut envoyer les commandes ligne par ligne, attendre la réponse ok, détecter les erreurs et arrêter le cycle en cas de problème.

L'ensemble de ces choix techniques permet d'obtenir une solution simple, cohérente et adaptée à la réalisation du prototype Sky Pharma.

5.5 Base de données SQLite

La **base de données SQLite** constitue le centre de **stockage** des informations du système. Elle permet de relier les données logiques, comme le nom d'un **médicament** ou son **stock**, avec les données physiques nécessaires au robot, comme les coordonnées X, Y et Z de l'emplacement.

Dans Sky Pharma, chaque **médicament** est représenté dans la base par un ensemble d'informations : son nom, sa description, sa catégorie, sa quantité disponible, son image éventuelle, ainsi que ses coordonnées dans la zone de **stockage**. Ces coordonnées sont très importantes, car elles permettent au logiciel de savoir vers quelle position le robot doit se déplacer lorsqu'un **médicament** est sélectionné.

La table des **médicaments** est donc directement utilisée par l'interface client pour afficher les produits disponibles. Lorsqu'un **médicament** est ajouté au panier puis commandé, le logiciel utilise son identifiant pour retrouver toutes les informations associées.

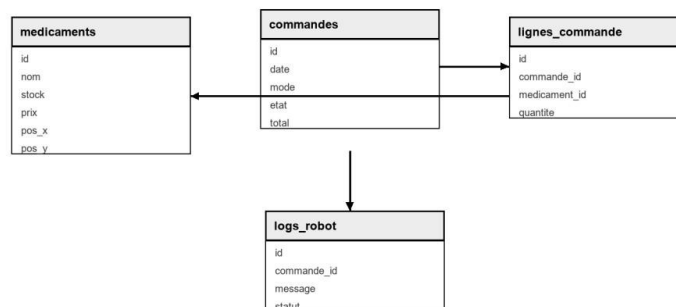
La base contient également une table des commandes. Cette table permet d'enregistrer chaque demande effectuée par un client ou par l'interface locale. Elle contient généralement la date de la commande, son état, son mode de récupération et éventuellement des informations sur le client ou la session. Une commande peut être en attente, en préparation, terminée ou en erreur.

Une table des lignes de commande permet de détailler le contenu de chaque commande. Elle relie une commande à un ou plusieurs **médicaments**. Cette structure est plus propre qu'un simple enregistrement unique, car elle permet à une même commande de contenir plusieurs produits.

Enfin, une table de logs robot peut être utilisée pour enregistrer les événements liés à l'exécution. Par exemple, le système peut enregistrer le lancement d'un cycle, l'envoi d'un **G-code**, la réception d'une réponse ok, une erreur **GRBL** ou la fin d'une préparation. Ces logs améliorent la **traçabilité** et facilitent le diagnostic en cas de problème.

Figure 5.4 - Modèle conceptuel de données de SkyPharma

Modèle conceptuel simplifié de la base de données



5.6 Organisation des tables de la base de données

La table `medicaments` constitue la table principale pour l’affichage et la gestion du **stock**. Elle contient les informations nécessaires au client, mais aussi les informations techniques utilisées par le robot. Par exemple, un **médicament** comme Doliprane peut être associé à une position précise dans la machine. Cette position est enregistrée sous forme de coordonnées X, Y et Z.

La table `commandes` permet de conserver l’historique des commandes validées. Chaque fois qu’un client confirme son panier, le logiciel crée une nouvelle commande. Cette commande reçoit un identifiant unique et un état initial. Cet état peut ensuite évoluer au cours du processus, par exemple de en attente vers en préparation, puis terminée.

La table `lignes_commande` permet de préciser les **médicaments** contenus dans une commande. Elle contient l’identifiant de la commande, l’identifiant du **médicament** et la quantité demandée. Cette organisation respecte la logique d’une base relationnelle et permet de garder une structure claire.

La table `logs_robot` complète le système en enregistrant les événements techniques. Elle permet de savoir quelles commandes ont été envoyées au robot, à quel moment, avec quel résultat. Dans un projet automatisé, cette **traçabilité** est importante, car elle permet de comprendre l’historique d’une opération et de détecter les causes d’une erreur.

Un exemple simplifié des tables peut être présenté comme suit :

Table	Rôle principal
<code>medicaments</code>	Stocker les médicaments, leurs informations et leurs coordonnées robot
<code>commandes</code>	Enregistrer les commandes passées par les clients ou localement
<code>lignes_commande</code>	Détailler les médicaments contenus dans chaque commande
<code>logs_robot</code>	Enregistrer les événements liés à l’exécution robotique

Tableau 5.1 - Organisation des tables de la base de données

Cette organisation permet de séparer proprement les données utilisateur, les données de **stock**, les données de commande et les données techniques.

5.7 Interface client distante

L’interface client distante est destinée à l’utilisateur final. Elle est accessible depuis une tablette ou un navigateur web connecté au même réseau Wi-Fi que le PC principal. L’utilisateur ouvre simplement l’adresse IP du PC hébergeant l’application **Streamlit**, puis accède à l’interface de commande.

Cette interface a été séparée de l’interface administrateur ou opérateur afin de protéger les fonctions techniques du système. Le client n’a pas besoin d’accéder aux réglages du robot, aux paramètres **GRBL**, aux logs détaillés ou aux outils de maintenance. Il doit uniquement pouvoir consulter les **médicaments**, sélectionner un produit, vérifier sa commande et confirmer.

L'interface client affiche donc les informations importantes de manière simple. Le client peut voir les **médicaments** disponibles, leur nom, leur description, leur disponibilité et éventuellement une image. L'objectif est de créer une expérience claire, proche d'une application de commande, tout en évitant les fonctions complexes qui pourraient perturber l'utilisateur.

Le choix d'une interface adaptée à une tablette est important. Dans un contexte de pharmacie automatisée, l'utilisateur doit pouvoir interagir rapidement avec le système. Les boutons doivent être visibles, les informations doivent être lisibles et le processus de commande doit être direct. Une interface trop chargée pourrait créer des erreurs de sélection ou ralentir l'expérience utilisateur.

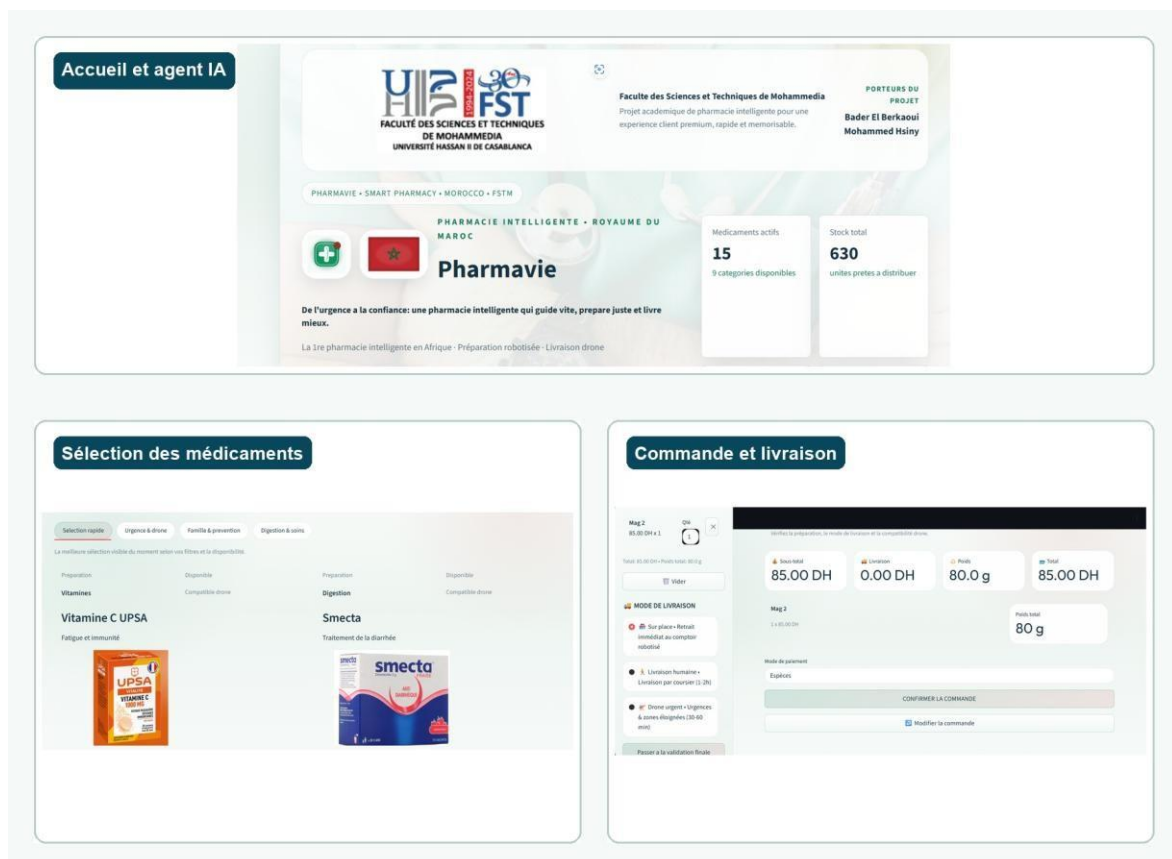


Figure 5.5 - Interface client distante avec agent IA et modes de livraison

5.8 Traitement logiciel après confirmation d'une commande

Lorsque le client confirme sa commande, le logiciel commence par récupérer le contenu du panier. Cette étape permet d'identifier les **médicaments** sélectionnés et les quantités demandées. Ensuite, le système interroge la base **SQLite** afin de vérifier que chaque **médicament** existe encore dans le **stock** et que sa quantité disponible est suffisante.

Après cette vérification, une nouvelle commande est créée dans la table commandes. Cette commande reçoit un identifiant et un état initial. Ensuite, pour chaque **médicament** sélectionné, le logiciel crée une ligne correspondante dans la table lignes_commande. Cette étape permet de conserver une trace précise du contenu de la commande.

Le système récupère ensuite les coordonnées du **médicament** dans la table **medicaments**. Ces coordonnées représentent la position physique du produit dans la zone de **stockage** du robot. C'est à ce moment que le lien entre la **base de données** et le robot devient essentiel. Le **médicament** n'est plus seulement une information affichée à l'écran ; il devient une position physique que le robot doit atteindre.

Le logiciel génère alors une séquence de **G-code**. Cette séquence comprend généralement un déplacement vers l'emplacement du **médicament**, une action sur l'axe Z pour permettre la récupération, un déplacement vers la zone de sortie, puis un dépôt du **médicament**. Le **G-code** est ensuite envoyé à Arduino/**GRBL** via pyserial. Après l'exécution, l'état de la commande est mis à jour dans la **base de données**.



Figure 5.6 - Diagramme de séquence du cycle de commande

5.9 Génération automatique du G-code

La génération automatique du **G-code** est une partie importante du projet, car elle permet de transformer les données de la base en mouvements réels du robot. Le logiciel ne se contente pas d'enregistrer une commande ; il construit une trajectoire adaptée au **médicament** sélectionné.

Le principe est le suivant : chaque **médicament** possède des coordonnées dans la **base de données**. Lorsque le client sélectionne un produit, le logiciel lit ces coordonnées puis construit une suite de commandes **G-code**. Ces commandes indiquent au robot où aller, à quelle hauteur descendre, où déposer le **médicament** et comment revenir à une position d'attente.

Un exemple simplifié de séquence peut être présenté ainsi :

```
G21
G90
G0 X120 Y80 F1000
G0 Z-20 F300
G0 Z0 F300
G0 X300 Y50 F1000
```

Dans cet exemple, G21 indique que les unités sont en millimètres, G90 active le mode absolu, G0 X120 Y80 déplace le robot vers l'emplacement du **médicament**, et les commandes sur l'axe Z permettent la descente puis la remontée du plateau.

Les valeurs utilisées dans le **G-code** ne sont pas fixes. Elles sont générées automatiquement à partir de la **base de données**. Ainsi, si le client choisit Doliprane, Vitamine C UPSA ou Smecta, le logiciel récupère les coordonnées correspondant à ce produit et produit une trajectoire adaptée.

Séquence de traitement d'une commande



Figure 5.7 - Diagramme de séquence du cycle de commande

5.10 Lien entre le logiciel et le robot

Le lien entre le logiciel et le robot est assuré par la communication série entre **Python** et **Arduino/GRBL**. Une fois que le **G-code** est généré, le logiciel ouvre le port série correspondant à l'Arduino, puis envoie les commandes ligne par ligne. Après chaque commande, il attend une réponse de **GRBL**.

Cette logique est importante pour éviter d'envoyer toutes les commandes sans contrôle. **GRBL** retourne généralement ok lorsque la commande est acceptée, error lorsqu'une commande est incorrecte, ou ALARM lorsqu'un problème de sécurité est détecté. Le logiciel doit donc interpréter ces réponses et arrêter le cycle en cas d'anomalie.

Le robot **H-Bot** reçoit finalement des commandes de déplacement classiques sous forme de coordonnées X, Y et Z. Comme le firmware **GRBL** a été adapté pour gérer la cinématique **H-Bot**, le logiciel peut travailler avec des coordonnées cartésiennes compréhensibles. **GRBL** se charge ensuite de transformer ces déplacements en mouvements combinés des moteurs A et B.

Ainsi, le logiciel reste simple du point de vue de la commande, tandis que la complexité cinématique est gérée au niveau embarqué. Cette séparation rend le système plus clair et plus facile à maintenir.

Séquence de traitement d'une commande



Figure 5.8 - Diagramme de séquence du cycle de commande

5.11 Interface locale de récupération

En plus de l'interface client distante, une interface locale a été développée pour la récupération sur site. Cette interface est utilisée directement au niveau de la station Sky Pharma. Elle peut être utilisée par un opérateur, par un responsable de démonstration ou par un utilisateur présent devant la machine.

L'objectif de cette interface locale est différent de celui de l'interface client distante. L'interface distante est centrée sur l'expérience client : consulter, choisir et confirmer. L'interface locale, quant à elle, est centrée sur l'exécution et la récupération immédiate. Elle permet de travailler directement avec le robot, de suivre l'état de la commande et de lancer une récupération locale.

Dans le cas d'une récupération locale, l'utilisateur peut sélectionner un **médicament** depuis l'interface locale. Le logiciel récupère alors les coordonnées du **médicament** dans la base **SQLite**, génère le **G-code** correspondant, puis envoie les commandes vers **GRBL**. Le robot exécute le cycle et dépose le **médicament** dans la zone de sortie. L'interface peut ensuite afficher un message indiquant que la récupération est terminée.

Cette interface locale est également utile pendant les tests. Elle permet de vérifier le comportement du robot, de lancer des cycles simples, de tester la communication et d'observer les réponses du système. Elle joue donc à la fois un rôle opérationnel et un rôle de supervision.

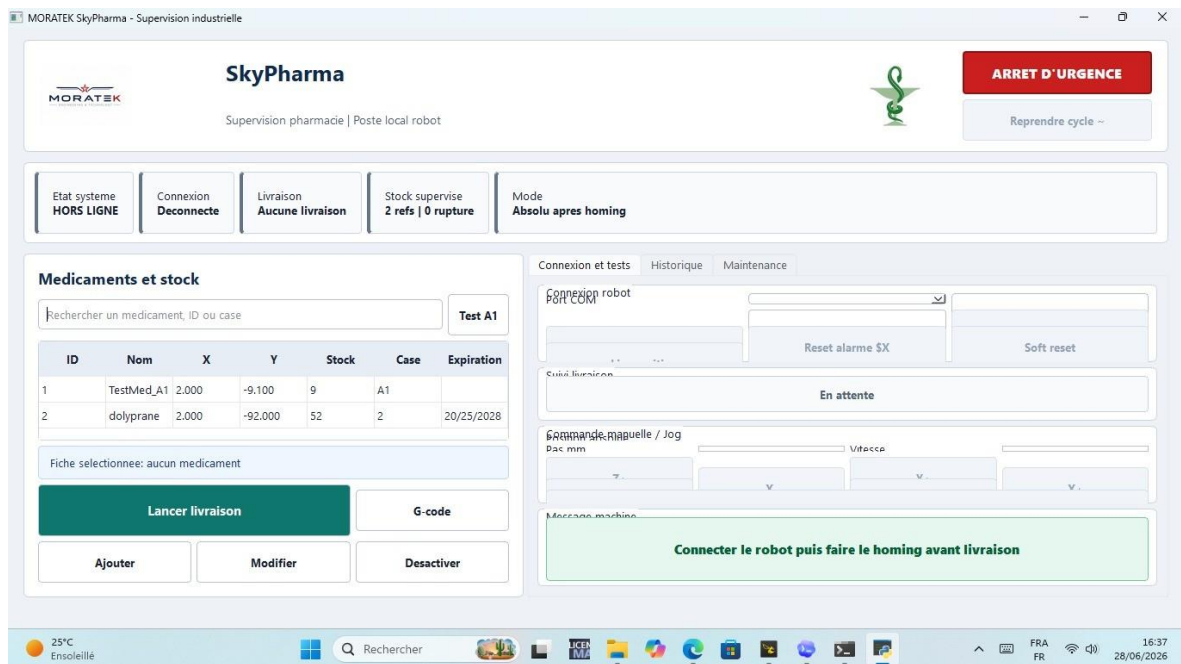


Figure 5.9 - Interface locale de supervision et de récupération automatisée

5.12 Processus de récupération locale

Le processus de récupération locale commence par l'ouverture de l'interface au niveau du PC de la station. L'utilisateur choisit un **médicament** parmi les produits disponibles. Comme pour l'interface distante, ces produits sont lus depuis **SQLite**, ce qui garantit que l'interface travaille avec les données réelles du système.

Après la sélection, le logiciel vérifie la disponibilité du **médicament**. Si le produit est disponible, une commande locale est créée. Cette commande peut être enregistrée dans la même table commandes, avec un mode de récupération local. Cette approche permet de conserver une **traçabilité** unique, que la commande soit passée à distance ou directement sur la station.

Le logiciel récupère ensuite les coordonnées du **médicament**, génère la trajectoire **G-code**, puis envoie les commandes à **GRBL**. Pendant l'exécution, l'interface locale peut afficher l'état du robot : en préparation, déplacement en cours, récupération, dépôt ou fin de cycle.

Une fois le **médicament** déposé dans la zone de sortie, l'interface affiche un message de fin de récupération. Le **stock** peut alors être mis à jour automatiquement et un log robot peut être enregistré.

Séquence de traitement d'une commande



Figure 5.10 - Diagramme de séquence du cycle de commande

5.13 Gestion du stock et traçabilité

La gestion du **stock** constitue une fonction essentielle du système logiciel. Lorsqu'un **médicament** est commandé et récupéré avec succès, la quantité disponible doit être mise à jour dans la **base de données**. Cette mise à jour permet d'éviter qu'un produit indisponible soit proposé à un autre utilisateur.

Dans le cas d'un prototype, la mise à jour peut être simple : après validation de la récupération, la quantité du **médicament** est décrétementée. Si la quantité atteint zéro, le produit peut être affiché comme indisponible dans l'interface client. Cette logique permet de rendre l'affichage plus cohérent avec l'état réel du **stock**.

La **traçabilité** est également importante. Chaque commande peut être enregistrée avec une date, un état, un mode de récupération et des logs techniques. Cette information permet de suivre le cycle complet d'une commande. Pour un projet de fin d'études, cet aspect est important car il montre que le système ne se limite pas à commander un moteur, mais intègre une logique de gestion et de suivi.

Les logs robot permettent aussi de diagnostiquer les erreurs. Par exemple, si une commande échoue, le système peut enregistrer si l'erreur vient de la communication série, d'un état **GRBL**, d'une limite déclenchée ou d'une commande incorrecte. Cette **traçabilité** améliore la maintenance et la compréhension du fonctionnement global.

5.14 Validation logicielle et amélioration par intelligence artificielle

Les captures des interfaces distante et locale constituent des éléments de validation fonctionnelle. Elles montrent la disponibilité du catalogue, le choix du mode de livraison, la confirmation de commande et la supervision du robot. Pour renforcer les performances du système, une amélioration par intelligence artificielle peut être intégrée sous forme d'agent d'assistance, de recommandation de médicaments compatibles, de priorisation des commandes urgentes et d'optimisation du chemin de récupération dans la matrice ASRS.

Cette évolution permettrait d'améliorer le temps de préparation, la qualité de service et la robustesse de la validation expérimentale, tout en conservant une supervision humaine pour les décisions sensibles liées à la santé.

Chapitre 6 : Réalisation du système de livraison par drone

6.1 Introduction

Dans le cadre du projet SkyPharma, la livraison par drone représente un module complémentaire permettant d'étendre les fonctionnalités de la plateforme au-delà de la distribution locale au sein de la pharmacie. Ce mode de livraison vise principalement les situations où l'accès rapide à un médicament devient critique, notamment dans les zones éloignées, les régions difficiles d'accès ou les cas d'urgence.

Le drone ne remplace pas les modes classiques de récupération. Il constitue une solution technologique supplémentaire intégrée à l'architecture globale du système. Ainsi, la plateforme SkyPharma propose trois modes de récupération : la récupération locale au niveau de la pharmacie, la livraison par un partenaire humain et la livraison autonome par drone.

Ce chapitre présente la conception, l'intégration matérielle, le principe de fonctionnement, le mécanisme d'accrochage du médicament, l'architecture électronique, la sécurité et la validation du sous-système drone utilisé dans le projet.

6.2 Objectif du système de livraison par drone

L'objectif principal du drone est d'assurer le transport d'un médicament léger depuis la pharmacie vers une position géographique définie par l'utilisateur. Le système doit être capable de recevoir une destination, d'exécuter une mission de vol, de transporter le médicament et de revenir vers son point de départ après la livraison.

Les objectifs spécifiques sont :

- réaliser une plateforme aérienne stable ;
- assurer une navigation assistée par GPS ;
- permettre le transport d'un médicament de faible masse ;
- intégrer le drone dans l'architecture globale de SkyPharma ;
- préparer la possibilité d'une mission autonome ;
- assurer l'accrochage et le largage du médicament à l'aide d'un mécanisme commandé ;
- garantir un minimum de sécurité pendant les essais.

6.3 Architecture générale du module drone



Figure 6.1 - Architecture générale du module de livraison par drone intégré à SkyPharma

Le flux général de fonctionnement est le suivant :

1. le client passe une commande via l'interface client ;
2. le serveur vérifie la disponibilité du médicament ;
3. le robot ASRS prépare le médicament ;
4. l'opérateur fixe le médicament au mécanisme d'accrochage du drone ;
5. les coordonnées GPS de destination sont préparées ;
6. le drone effectue la mission de livraison ;
7. le mécanisme de largage libère le médicament ;
8. l'état de la mission est mis à jour dans le système.

6.4 Choix de la plateforme aérienne

6.4.1 Châssis F450



Figure 6.2 - Châssis F450 utilisé pour la réalisation du drone

Le choix du F450 est justifié par :

- une structure mécanique simple et robuste ;
- une bonne accessibilité aux composants ;
- une compatibilité avec plusieurs contrôleurs de vol ;
- une facilité de maintenance ;
- une capacité suffisante pour transporter une charge légère.

6.4.2 Contrôleur de vol

Le contrôleur de vol représente l'élément central du drone. Il assure la stabilisation, la lecture des capteurs, la commande des moteurs et l'exécution des modes de vol.

Dans le cadre du projet, le drone a été étudié autour d'un contrôleur de type APM ArduPilot. Ce contrôleur permet l'utilisation d'un logiciel de planification de mission tel que Mission Planner afin de définir des points GPS et d'exécuter un trajet automatique.



Figure 6.3 - Contrôleur de vol APM ArduPilot

Le rôle du contrôleur de vol est de :

- stabiliser le drone en roulis, tangage et lacet ;
- gérer les signaux provenant du récepteur radio ;
- envoyer les commandes PWM vers les ESC ;
- exploiter les données du GPS ;
- exécuter les modes de vol assistés ou autonomes ;
- assurer le retour automatique en cas de perte de signal ou de batterie faible.

6.4.3 Radiocommande FlySky FS-i6

La radiocommande utilisée est une FlySky FS-i6. Elle permet le pilotage manuel du drone lors des essais, la sélection des modes de vol et l'intervention en cas de besoin.



Figure 6.4 - Radiocommande FlySky FS-i6 utilisée pour le pilotage manuel

La commande manuelle reste importante dans un prototype académique, car elle permet de sécuriser les premiers essais et de reprendre le contrôle du drone en cas d'anomalie.

6.4.4 Motorisation

Le drone est équipé de moteurs brushless SunnySky 1400 KV. Le coefficient KV indique le nombre de tours par minute par volt appliqué au moteur à vide. Un moteur de 1400 KV est adapté à des hélices de taille moyenne et à une alimentation de type LiPo 3S.



Figure 6.5 - Moteur brushless SunnySky 1400 KV

6.4.5 Hélices



Figure 6.6 - Hélices de 8 pouces utilisées sur le drone

6.4.6 Batterie

L'alimentation du drone est assurée par une batterie LiPo 3S de tension nominale 11,1 V, de capacité 6500 mAh et de taux de décharge 80C.



Figure 6.7 - Batterie LiPo 3S 6500 mAh utilisée pour l'alimentation du drone

Le courant maximal théorique peut être estimé par :

$$I_{\max} = C \times Q$$

Avec :

- $I_{\max}$: courant maximal théorique ;
- C : taux de décharge ;
- Q : capacité de la batterie en ampères-heures.

Dans notre cas :

$$I_{\max} = 80 \times 6,5 = 520 \text{ A}$$

Cette valeur reste théorique. En pratique, le courant réellement utilisé dépend de la motorisation, des ESC, de la masse totale du drone, de la charge transportée et du profil de vol.

6.5 Mécanisme d'accrochage et de largage du médicament

6.5.1 Objectif du mécanisme

Afin de permettre au drone de transporter un médicament, un mécanisme d'accrochage et de largage a été développé. Ce mécanisme a pour rôle de maintenir le médicament pendant la phase de vol, puis de le libérer au niveau de la zone de livraison.

Le choix d'un mécanisme commandé par servomoteur permet d'obtenir une solution simple, légère et facilement contrôlable. Le servomoteur peut être commandé par un signal PWM provenant du contrôleur embarqué ou d'un microcontrôleur auxiliaire.

6.5.2 Principe mécanique

Le mécanisme est basé sur une pièce mécanique conçue spécifiquement pour assurer deux fonctions principales :

- maintenir le médicament pendant le transport ;
- libérer le médicament lorsque le servomoteur change de position angulaire.

Le principe de fonctionnement repose sur une rotation contrôlée du bras du servomoteur. Lorsque le servo est en position fermée, la pièce bloque le médicament ou son support. Lorsque le servo tourne vers la position ouverte, la pièce libère le support et permet le largage.

6.5.3 États du mécanisme

Tableau 6.1 - États du mécanisme d'accrochage

État	Description
Position fermée	Le médicament est maintenu mécaniquement. Le drone peut décoller et transporter la charge.
Position ouverte	Le servomoteur libère la pièce de maintien, ce qui permet le dépôt ou le largage du médicament.

6.5.4 Intégration mécanique

La pièce du mécanisme a été conçue de manière à être légère afin de ne pas augmenter excessivement la masse embarquée. Elle doit également être suffisamment rigide pour supporter les vibrations du drone pendant le vol.

Les principaux critères de conception sont :

- masse réduite ;
- rigidité suffisante ;
- simplicité d'impression 3D ;
- fixation facile sur le châssis ;
- compatibilité avec le servomoteur ;
- sécurité du médicament pendant le transport.



Figure 6.8 - Pièce mécanique développée pour le système d'accrochage

6.5.5 Commande du servomoteur

Le servomoteur est commandé par un signal PWM. Le principe consiste à définir deux positions angulaires : une position de verrouillage et une position de libération.

Tableau 6.2 - Commande logique du servomoteur

Action	Angle servo	Effet mécanique
Verrouillage	0°	Le médicament est maintenu.
Largage	90°	Le médicament est libéré.

6.5.6 Séquence de livraison avec largage

La séquence complète de livraison avec le mécanisme d'accrochage est la suivante :

1. l'opérateur place le médicament dans le support ;
2. le servomoteur est positionné en mode verrouillé ;
3. le drone décolle ;
4. le drone atteint la zone de livraison ;
5. le servomoteur passe en position ouverte ;
6. le médicament est libéré ;
7. le drone revient vers son point de départ.

Chapitre 6 : Réalisation du système de livraison par drone

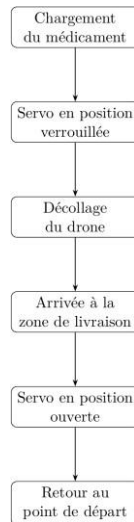


Figure 6.9 - Séquence simplifiée du mécanisme d'accrochage et de largage commandé par servomoteur

6.6 Architecture électronique du drone

L'architecture électronique du drone est composée de plusieurs éléments interconnectés : batterie, contrôleur de vol, ESC, moteurs brushless, GPS, récepteur radio et servomoteur de largage.

Schéma électronique général du drone de livraison

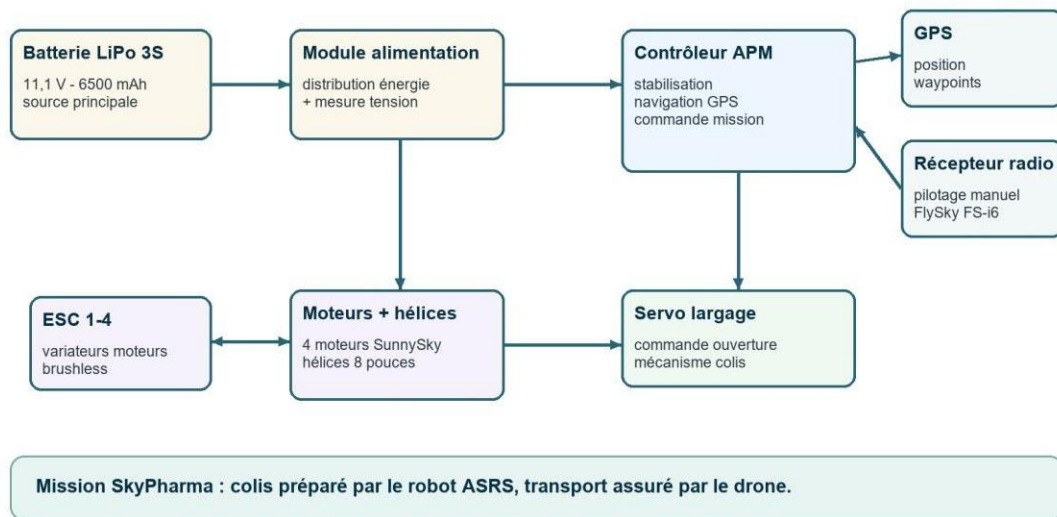


Figure 6.10 - Schéma électronique général du drone

Tableau 6.3 - Rôle des principaux composants du drone

Composant	Rôle
Frame F450	Support mécanique du drone et fixation des composants.
Contrôleur de vol APM	Stabilisation, traitement des capteurs et gestion des modes de vol.
GPS	Localisation du drone et navigation par points GPS.
ESC	Conversion des signaux de commande en puissance pour les moteurs.
Moteurs SunnySky 1400 KV	Génération de la poussée nécessaire au vol.
Hélices 8 pouces	Transformation de la rotation des moteurs en poussée aérodynamique.
Batterie LiPo 3S 6500 mAh	Source d'énergie principale du drone.
Radiocommande FlySky FS-i6	Commande manuelle et sélection des modes de vol.
Servomoteur	Commande du mécanisme d'accrochage et de largage du médicament.

6.7 Principe de fonctionnement de la mission

La mission de livraison par drone est pensée comme une extension du système SkyPharma. Le drone intervient après la préparation du médicament par le robot ASRS.

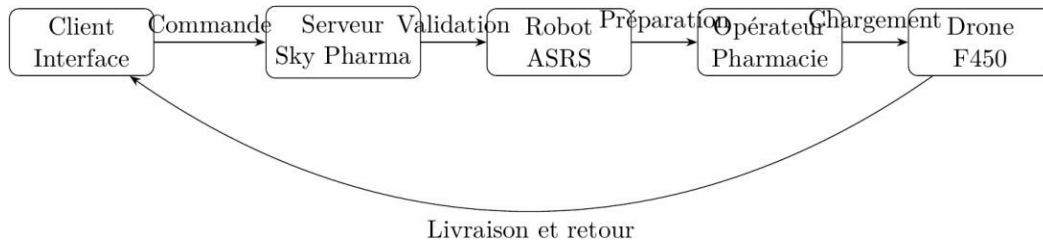


Figure 6.11 - Diagramme simplifié de la séquence de livraison par drone

Le scénario de fonctionnement est :

1. le client sélectionne le mode de livraison par drone ;
2. le serveur récupère les coordonnées du client ;
3. le système vérifie la disponibilité du médicament ;
4. le robot ASRS prépare le médicament ;
5. l'opérateur charge le médicament sur le drone ;
6. le mécanisme d'accrochage est verrouillé ;
7. la mission GPS est configurée ;
8. le drone décolle ;
9. le drone rejoint la destination ;
10. le mécanisme de largage est activé ;
11. le médicament est livré ;
12. le drone retourne à son point de départ.

6.8 Sécurité du vol

La sécurité représente un aspect essentiel dans tout système aérien autonome. Dans le cadre de ce projet académique, la livraison par drone est étudiée comme un prototype expérimental.

Tableau 6.4 - Risques et mesures de prévention

Risque	Mesure de prévention
Perte de signal radio	Activer le mode Fail-Safe et Return To Launch.
Batterie faible	Vérifier la tension avant décollage et limiter la durée de mission.
Perte GPS	Ne pas lancer de mission autonome sans GPS stable.
Surcharge	Limiter la masse du médicament transporté.
Instabilité	Vérifier le centre de gravité et la fixation de la charge.
Mauvais largage	Tester le servomoteur au sol avant le décollage.

Risque	Mesure de prévention
Collision	Réaliser les essais dans une zone dégagée.

6.9 Validation expérimentale

La validation expérimentale du drone est réalisée progressivement afin de réduire les risques. Elle commence par les tests mécaniques et électriques, puis passe aux essais de stabilisation, aux tests de navigation et aux essais du mécanisme d'accrochage.

Tableau 6.5 - Plan de validation du drone

Test	Objectif	Résultat attendu
Test moteur	Vérifier les quatre moteurs	Rotation correcte
Calibration radio	Assurer la lecture des commandes	Commandes stables
Calibration capteurs	Corriger les mesures internes	Capteurs opérationnels
Test servomoteur	Vérifier ouverture et fermeture	Mécanisme fonctionnel
Test charge	Vérifier le maintien du médicament	Charge stable
Test décollage	Vérifier la stabilité initiale	Décollage contrôlé
Test GPS	Vérifier la localisation	Position stable
Test RTL	Vérifier le retour automatique	Retour au point de départ

6.10 Limites et perspectives

Le prototype réalisé permet de valider le principe d'une livraison par drone intégrée à la plateforme SkyPharma. Toutefois, plusieurs limites doivent être prises en considération : autonomie limitée, sensibilité au vent, charge utile réduite, dépendance au GPS, contraintes réglementaires et nécessité de sécuriser davantage le mécanisme de largage.

Les perspectives principales sont :

- amélioration du mécanisme de largage ;
- ajout d'un capteur confirmant la présence ou l'absence du médicament ;
- ajout d'une caméra embarquée ;
- utilisation d'un contrôleur de vol plus récent ;
- communication 4G ou 5G ;
- suivi temps réel de la mission ;
- détection d'obstacles ;
- optimisation de l'autonomie.

6.11 Conclusion

Ce chapitre a présenté la réalisation du module de livraison par drone intégré à la plateforme SkyPharma. Le drone constitue un moyen complémentaire de livraison permettant d'envisager l'accès rapide aux médicaments dans des contextes particuliers.

La plateforme réalisée repose sur un châssis F450, des moteurs brushless SunnySky 1400 KV, une batterie LiPo 3S 6500 mAh, des hélices de 8 pouces, une radiocommande FlySky FS-i6, un contrôleur de vol compatible avec la planification de missions et un mécanisme d'accrochage commandé par servomoteur.

L'ajout du mécanisme d'accrochage et de largage permet de relier la mission aérienne à l'objectif réel du projet, à savoir le transport et la livraison d'un médicament. Même si le système reste un prototype académique, il démontre la faisabilité d'une intégration entre robotique de stockage, systèmes embarqués, automatisation et livraison autonome.

CONCLUSION GÉNÉRALE

Ce projet a permis de concevoir une plateforme complète de pharmacie autonome autour d'un robot **ASRS**, d'une commande électronique, d'un logiciel de gestion et d'une interface de supervision. L'approche suivie montre l'intérêt de l'intégration entre génie électrique, contrôle industriel, mécanique, informatique et logistique de santé.

La version actuelle du rapport conserve le contenu technique développé dans le document initial tout en améliorant sa structure, sa lisibilité, ses figures, ses tableaux et sa pagination. Les prochaines modifications pourront porter sur l'ajout de mesures expérimentales réelles, de captures définitives SolidWorks et d'une bibliographie plus complète.

ANNEXES ET BIBLIOGRAPHIE

Réf.	Source	Objet
[1]	Documentation Arduino	Arduino UNO et environnement de développement
[2]	Documentation GRBL	Commandes G-code et configuration CNC
[3]	Texas Instruments / Pololu	Driver DRV8825 et recommandations thermiques
[4]	Documentation Python / SQLite	Base de données et communication série
[5]	Documentation interne du projet	Code, photos, essais et fichiers SkyPharma
[6]	Python Software Foundation	Logo Python utilisé dans la cartographie technologique
[7]	Wikimedia Commons - Arduino Logo	Logo Arduino et photographie Arduino UNO
[8]	SQLite / Wikimedia Commons	Logo SQLite et référence base embarquée
[9]	Wikimedia Commons - Streamlit	Logo Streamlit pour la couche interface
[10]	Wikimedia / SolidWorks	Logo SolidWorks pour la couche CAO

Annexe A : Sources et références consultées

Tableau 6.6 - Sources et références consultées